Universidade Federal do Amazonas — UFAM
Instituto de Ciências Exatas e Tecnologia — ICET
Curso de Sistemas de Informação

SLOW DOWN

Documentação do Sistema

Aplicativo de apoio ao cuidado contra o burnout ocupacional

Versão 2.0 — documentação atualizada conforme o software entregue (MVP)

Equipe
Evelly Rebeca · Felipe Rangel Silvestre · Kaio Sobral Moreira
Marcelo Barros · Marcos Oliveira · Nádia Leão

Engenharia de Software — Prof. Walter Viana
Itacoatiara — AM, novembro de 2026

Histórico de revisões

Versão	Data	Descrição	Autor
1.0	26/04/2026	Especificação inicial (Plano de Trabalho, Design Thinking e Backlog — TP1).	Equipe
1.1	06/2026	Arquitetura C4, rastreabilidade e casos de teste (TP2 e TP3).	Equipe
2.0	12/11/2026	Documentação reescrita a partir do código do MVP (TP4): requisitos, regras de negócio, diagramas UML, arquitetura e padrões de projeto conferidos contra o repositório.	Felipe Rangel

Sumário

1. Introdução

1.1. Descrição Geral do Sistema

1.2. Descrição dos Usuários

2. Requisitos Gerais do Sistema

2.1. Requisitos Funcionais

2.2. Requisitos Não-Funcionais

2.3. Regras de Negócio

3. Diagramas UML

3.1. Diagramas de Casos de Uso

3.2. Especificação dos Casos de Uso

3.3. Diagramas de Sequência

3.4. Diagramas de Atividades

3.5. Diagrama de Classes

4. Arquitetura

5. Padrões de Projeto

6. Prototipação das Telas

A. Apêndice A — Gerência de configuração e versionamento

Como ler este documento. Todo requisito, regra e diagrama aponta para o arquivo do repositório onde está implementado (caminhos relativos a `4-MVP/mvp/`). Quando uma funcionalidade ainda é simulada no MVP, isso está dito explicitamente, para que a documentação corresponda ao software apresentado.

1. Introdução

1.1. Descrição Geral do Sistema

O **SlowDown** é um aplicativo móvel de prevenção ao burnout ocupacional, fenômeno reconhecido pela OMS desde 2019. Ele reúne, em um só lugar, ferramentas de autocuidado que o trabalhador pode usar em pausas curtas: registro diário do estado emocional, termômetro emocional com sugestões, monitoramento da frequência cardíaca, meditações guiadas, sons relaxantes, um assistente de apoio por chat e uma camada de gamificação (pet virtual, missões diárias com XP e emblemas, minijogos) que transforma o cuidado em hábito.

O sistema é composto por duas aplicações que conversam por HTTP/JSON:

- **Aplicativo móvel** em Flutter (pasta `frontend/`), com foco em Android. Contém 42 telas e guarda localmente (SharedPreferences) os dados de uso do dia a dia, o que permite usar a maior parte das funções sem internet.
- **API REST** em Node.js + Express (pasta `backend/`), organizada em MVC, que valida o token do usuário no Firebase e mantém o cadastro do usuário no MySQL.

A autenticação (e-mail/senha, Google e recuperação de senha) é feita pelo **Firebase Authentication**; o MySQL guarda apenas os dados do usuário dentro do app (id interno, nome, e-mail e plano).

Escopo do MVP entregue

O quadro abaixo resume o que funciona de ponta a ponta e o que, nesta versão, ainda é simulado na interface. A mesma classificação é usada na tabela de requisitos (seção 2.1).

Situação	Funcionalidades
Implementado	Cadastro e login por e-mail/senha; login com Google; recuperação de senha; sincronização do usuário com o MySQL (`POST /auth/sync`); logout; integração com o Spotify (requer o app Spotify instalado); meditação guiada com temporizador e respiração; minijogos.
Implementado (dados locais)	Registro, histórico e termômetro emocional; pet virtual e personalização; missões, XP, nível e emblemas; onboarding e perfil inicial; preferências de notificações, acessibilidade e modo offline.
Parcial / simulado	Código OTP (aceita o código fixo `123456`); leitura de BPM (valor simulado); assistente de apoio (respostas por regras locais, sem IA); sons/sleepcasts (player simulado, sem áudio); assinatura Premium (sem gateway); conexão com Apple Health/Google Fit, envio de push e downloads offline (apenas estado salvo); relatório (dados de exemplo para meditação/BPM); comunidade (dados locais).

Fora do escopo: o app não substitui diagnóstico ou tratamento clínico e não terá versão web completa (Plano de Trabalho, seção 2.1.2).

1.2. Descrição dos Usuários

O sistema tem um único tipo de usuário humano, diferenciado pelo estado de autenticação e pelo plano. Não há perfil de administrador nesta versão.

Usuário	Descrição	O que pode fazer
Visitante	Pessoa que abriu o app e ainda não entrou na conta.	Configurar a acessibilidade inicial, cadastrar-se, entrar (e-mail/senha ou Google) e recuperar a senha.

Usuário	Descrição	O que pode fazer
Usuário autenticado — plano padrão	Usuário com conta no Firebase sincronizada no MySQL (`users.plano = 'padrao'` , valor padrão).	Todas as funcionalidades do app: monitoramento emocional, meditação, sons, pet, missões, jogos, assistente, configurações.
Usuário Premium	Usuário que ativou um plano mensal ou anual na tela Premium.	As mesmas funções; a tela mostra o plano ativo e os dias restantes. No MVP o status Premium é salvo apenas no dispositivo.

Os usuários foram caracterizados no Design Thinking por três personas, que continuam orientando as prioridades:

Persona	Perfil	Necessidades atendidas no app
Geilson Alves (Premium)	Trabalhador de campo, esforço físico, conectividade instável.	Meditação curta (US-01), dados salvos no aparelho para uso sem rede (US-19), Spotify (US-02), registro emocional (US-06).
Arleson Mota (Padrão)	Gestor de equipe sob pressão por resultados.	Monitoramento de BPM (US-04), relatório (US-10), comunidade/pessoa próxima (US-11).
Ana Beatriz Lemos (Acessibilidade)	Profissional de tecnologia com alta carga mental e dificuldade de desconectar.	Acessibilidade e voz (US-05), pet e missões (US-03, US-13), assistente (US-08), notificações (US-15).

2. Requisitos Gerais do Sistema

2.1. Requisitos Funcionais

Cada requisito foi derivado de uma História de Usuário (US) do Backlog do Produto e conferido no código. A coluna *Situação* informa o grau de implementação no MVP.

ID	Requisito	US	Situação	Evidência no código
RF01	Exibir, na abertura, a configuração de acessibilidade com opção de comandos de voz antes do login.	US-05	**Parcial / simulado**	`accessibility_screen.dart`
RF02	Cadastrar conta com nome, e-mail, senha, confirmação de senha e aceite dos Termos de Uso.	US-16	**Implementado**	`register_screen.dart`, `AuthNotifier.registar()`
RF03	Verificar código de confirmação de 6 dígitos com contagem regressiva de 15 minutos.	US-16	**Parcial / simulado**	`otp_verification_screen.dart`, `POST /auth/verify-otp`
RF04	Autenticar o usuário com e-mail e senha.	US-16	**Implementado**	`login_screen.dart`, `AuthNotifier.login()`
RF05	Autenticar ou cadastrar o usuário com a conta Google.	US-16	**Implementado**	`AuthService.loginComGoogle()`
RF06	Enviar link de redefinição de senha para o e-mail informado.	US-16	**Implementado**	`forgot_password_screen.dart`
RF07	Sincronizar o usuário autenticado com o banco: criar no primeiro acesso e devolver os dados nos acessos seguintes.	US-16	**Implementado**	`POST /auth/sync`, `authController.js`, `userModel.js`
RF08	Manter a sessão (token, id e nome) no dispositivo e encerrá-la ao sair.	US-16	**Implementado**	`session_manager.dart`, `menu_screen.dart`
RF09	Apresentar onboarding guiado no primeiro acesso e configurar perfil inicial (apelido e objetivo).	US-14	**Implementado (dados locais)**	`onboarding_screen.dart`, `profile_setup_screen.dart`
RF10	Registrar o estado emocional do dia (escala 1–10, emoji, cor e nota).	US-06	**Implementado (dados locais)**	`emotional_record_screen.dart`, `emotional_entry_storage.dart`
RF11	Listar o histórico emocional com média, melhor e pior registro e permitir limpá-lo.	US-06, US-10	**Implementado (dados locais)**	`emotional_history_screen.dart`
RF12	Exibir termômetro emocional: média e tendência semanal, gráficos de 7 e 30 dias e sugestões.	US-07	**Implementado (dados locais)**	`emotional_thermometer_screen.dart` (`EmotionalInsights`)
RF13	Monitorar a frequência cardíaca, classificá-la (abaixo do normal, normal, elevada) e mostrar o histórico semanal.	US-04	**Parcial / simulado**	`monitor_screen.dart`, `validador_bpm.dart`
RF14	Exibir relatório de bem-estar por semana ou mês (humor, meditação e BPM).	US-10	**Parcial / simulado**	`report_screen.dart`

ID	Requisito	US	Situação	Evidência no código
RF15	Conversar com um assistente de apoio com três estilos de resposta e encaminhar para o CVV em mensagens de risco.	US-08	**Parcial / simulado**	`chatbot_screen.dart`
RF16	Listar e executar sessões de meditação guiada com temporizador e instruções de respiração.	US-01	**Implementado**	`meditation_screen.dart`, `meditation_session_screen.dart`
RF17	Reproduzir sons ambiente e sleepcasts com temporizador de sono.	US-18	**Parcial / simulado**	`music_screen.dart`, `sleepcasts_screen.dart`
RF18	Conectar ao Spotify e controlar a reprodução (tocar, pausar, avançar).	US-02	**Implementado**	`spotify_screen.dart`
RF19	Gerenciar conteúdo offline: itens baixados, espaço ocupado, download só por Wi-Fi e sincronização automática.	US-02, US-19	**Parcial / simulado**	`offline_screen.dart`
RF20	Cuidar do pet virtual (alimentar, descansar e brincar), com status de fome, felicidade e energia.	US-03	**Implementado (dados locais)**	`pet_screen.dart`
RF21	Personalizar o pet (raça, chapéu e óculos).	US-03	**Implementado (dados locais)**	`pet_skin_screen.dart`
RF22	Cumprir missões diárias de autocuidado, acumulando XP, nível e sequência de dias.	US-13	**Implementado (dados locais)**	`missions_screen.dart` (`GamificationStatus`)
RF23	Desbloquear emblemas digitais ao atingir metas.	US-12	**Implementado (dados locais)**	`missions_screen.dart` (`kBadgeCatalog`)
RF24	Jogar minijogos antiestresse (Quiz, Corrida e Damas).	US-09	**Implementado**	`games_screen.dart`, `quiz_screen.dart`, `race_screen.dart`, `checkers_screen.dart`
RF25	Configurar notificações: permissão, horário do lembrete e categorias.	US-15	**Parcial / simulado**	`notifications_screen.dart`
RF26	Assinar ou cancelar o plano Premium (mensal ou anual).	US-17	**Parcial / simulado**	`premium_screen.dart`
RF27	Visualizar o perfil do usuário.	US-16	**Parcial / simulado**	`profile_screen.dart`
RF28	Configurar preferências gerais e de acessibilidade (comandos de voz, texto grande, alto contraste).	US-05	**Parcial / simulado**	`settings_screen.dart`
RF29	Conectar Apple Health ou Google Fit e registrar a última sincronização.	US-04	**Parcial / simulado**	`health_sync_screen.dart`
RF30	Interagir na comunidade (perfil, feed de humor e mensagens).	US-11	**Parcial / simulado**	`social_screen.dart`

2.2. Requisitos Não-Funcionais

ID	Categoria	Requisito	Evidência
RNF01	Portabilidade	Aplicativo em Flutter (Dart SDK ^3.12), com Android como plataforma-alvo do MVP. O mesmo código gera iOS, Web e desktop.	`frontend/pubspec.yaml`
RNF02	Tecnologia do servidor	API em Node.js 18+ com Express 5, escutando em `0.0.0.0` na porta 3000 (configurável por `PORT`) para aceitar emulador e dispositivos da rede.	`backend/src/server.js`
RNF03	Persistência	MySQL 8 com charset utf8mb4; acesso por pool de até 10 conexões.	`database.sql`, `config/database.js`
RNF04	Segurança — autenticação	Senhas nunca chegam ao backend: a autenticação é do Firebase. Rotas protegidas exigem `Authorization: Bearer <ID token>` validado pelo Firebase Admin SDK; sem token válido, a API responde 401.	`middlewares/authMiddleware.js`
RNF05	Segurança — credenciais	Chaves e senhas ficam fora do repositório: `.env` e `firebase-service-account.json` estão no `.gitignore`; `.env.example` serve de modelo.	`.gitignore`, `backend/.env.example`
RNF06	Desempenho	O cliente HTTP define tempo limite de 10 s para conexão e para resposta.	`src/services/api_client.dart`
RNF07	Disponibilidade sem rede	Registros emocionais, pet, missões, histórico do chat e preferências são gravados no aparelho e funcionam sem conexão.	`SharedPreferences` (ver seção 3.5)
RNF08	Usabilidade e acessibilidade	Interface em português, tela de acessibilidade na abertura, opções de texto grande, alto contraste e comandos de voz; erros e confirmações exibidos em SnackBar.	`accessibility_screen.dart`, `settings_screen.dart`
RNF09	Manutenibilidade	Backend em MVC; URL da API centralizada em um único arquivo; regras de negócio isoladas em validadores puros.	`ApiConfig`, `lib/src/utils/`
RNF10	Testabilidade	65 testes automatizados (unidade e widget) cobrindo US-04, US-06 e US-16, todos aprovados.	`frontend/test/`, `resultado_testes.txt`
RNF11	Privacidade (LGPD)	Cadastro exige aceite dos Termos de Uso e da Política de Privacidade.	`4-MVP/docs/termos-de-uso.md`, `politica-de-privacidade.md`
RNF12	Segurança emocional	O assistente prioriza o encaminhamento ao CVV (188) sempre que detecta mensagem de risco, antes de qualquer outra resposta.	`chatbot_screen.dart`
RNF13	Configurabilidade	Parâmetros de banco, porta e credencial do Firebase lidos de variáveis de ambiente (dotenv).	`backend/.env.example`

2.3. Regras de Negócio

Regras extraídas das classes de validação e de modelo. As regras RN01, RN04, RN07 e RN08 são verificadas pelos testes automatizados.

ID	Regra	Onde é aplicada
RN01	A senha deve ter no mínimo 6 caracteres e pelo menos um número.	`ValidadorAuth.validarSenha()`

ID	Regra	Onde é aplicada
RN02	Nome é obrigatório; e-mail deve ter formato válido; a confirmação deve ser igual à senha; os Termos devem ser aceitos para cadastrar.	`ValidadorAuth`, `register_screen.dart`
RN03	O login exige e-mail e senha preenchidos; credenciais inválidas exibem “E-mail ou senha incorretos.”; e-mail já cadastrado exibe “Este e-mail já está em uso.”.	`login_screen.dart`, `AuthNotifier._tratarErroFirebase()`
RN04	O código de verificação tem 6 dígitos e expira em 15 minutos (900 s). No MVP o código válido é fixo: `123456`. Cadastro com Google dispensa o código.	`validarCodigoVerificacao()`, `authController.verifyOtp`
RN05	Cada usuário é identificado pelo `firebase_uid` (único). Na primeira sincronização é criado no MySQL com plano `padrao`; o e-mail também é único.	`authController.syncUser`, `database.sql`
RN06	Toda rota protegida da API exige token do Firebase válido; caso contrário retorna 401.	`authMiddleware.js`
RN07	Registro emocional: a escala é obrigatória e vai de 1 a 10; emoji e cor são opcionais, mas, se informados, devem pertencer ao conjunto de 9 emojis e à paleta de 6 cores; a nota tem no máximo 500 caracteres.	`ValidadorRegistroEmocional`
RN08	Frequência cardíaca: abaixo de 60 BPM = abaixo do normal; 60 a 100 = normal; acima de 100 = elevada. Leituras fora do normal geram alerta. O intervalo de atualização deve ficar entre 5 e 60 s.	`ValidadorBpm`
RN09	Missões: cada missão vale de 15 a 25 XP e só pode ser concluída uma vez por dia; as missões reiniciam na virada do dia; nível = ⌊XP / 100⌋ + 1; a sequência aumenta quando há conclusão em dias consecutivos e volta a 1 se um dia for pulado.	`GamificationStatus`
RN10	Emblemas: Primeira missão (1 missão), 10 missões cumpridas, Sequência de 3 dias, Sequência de 7 dias e Nível 5 alcançado. São verificados após cada missão concluída.	`kBadgeCatalog`
RN11	Pet: fome, felicidade e energia variam de 0 a 100 e caem a cada 30 s (−3, −2 e −1,5). Alimentar: +20 fome e +5 felicidade (bloqueado se fome = 100). Descansar: +25 energia (bloqueado se energia = 100). Brincar exige energia ≥ 10. Humor: média ≥ 70 feliz, ≥ 40 neutro, abaixo disso triste.	`PetStatus`, `pet_screen.dart`
RN12	Premium: plano mensal vale 30 dias e anual 365 dias a partir da assinatura; ao cancelar, o acesso Premium é removido.	`PremiumStatus.subscribe()/cancel()`
RN13	Assistente: mensagens com palavras de risco (ex.: “quero morrer”, “não aguento mais”) disparam o cartão de ajuda com o CVV 188; o histórico guarda no máximo as 200 últimas mensagens.	`_LocalChatEngine`, `ChatStorage`
RN14	Notificações: lembrete padrão às 20:00; categorias meditação, registro emocional, missões e premium ativadas e social desativada por padrão.	`NotificationSettings`
RN15	Offline: por padrão, downloads só ocorrem em Wi-Fi.	`OfflineSettings.wifiOnly`

3. Diagramas UML

3.1. Diagramas de Casos de Uso

Os casos de uso foram separados em dois diagramas para legibilidade. Atores: **Visitante** (não autenticado), **Usuário Autenticado** (especialização do Visitante) e os sistemas externos **Firebase Authentication**, **Google Sign-In** e **Spotify**.

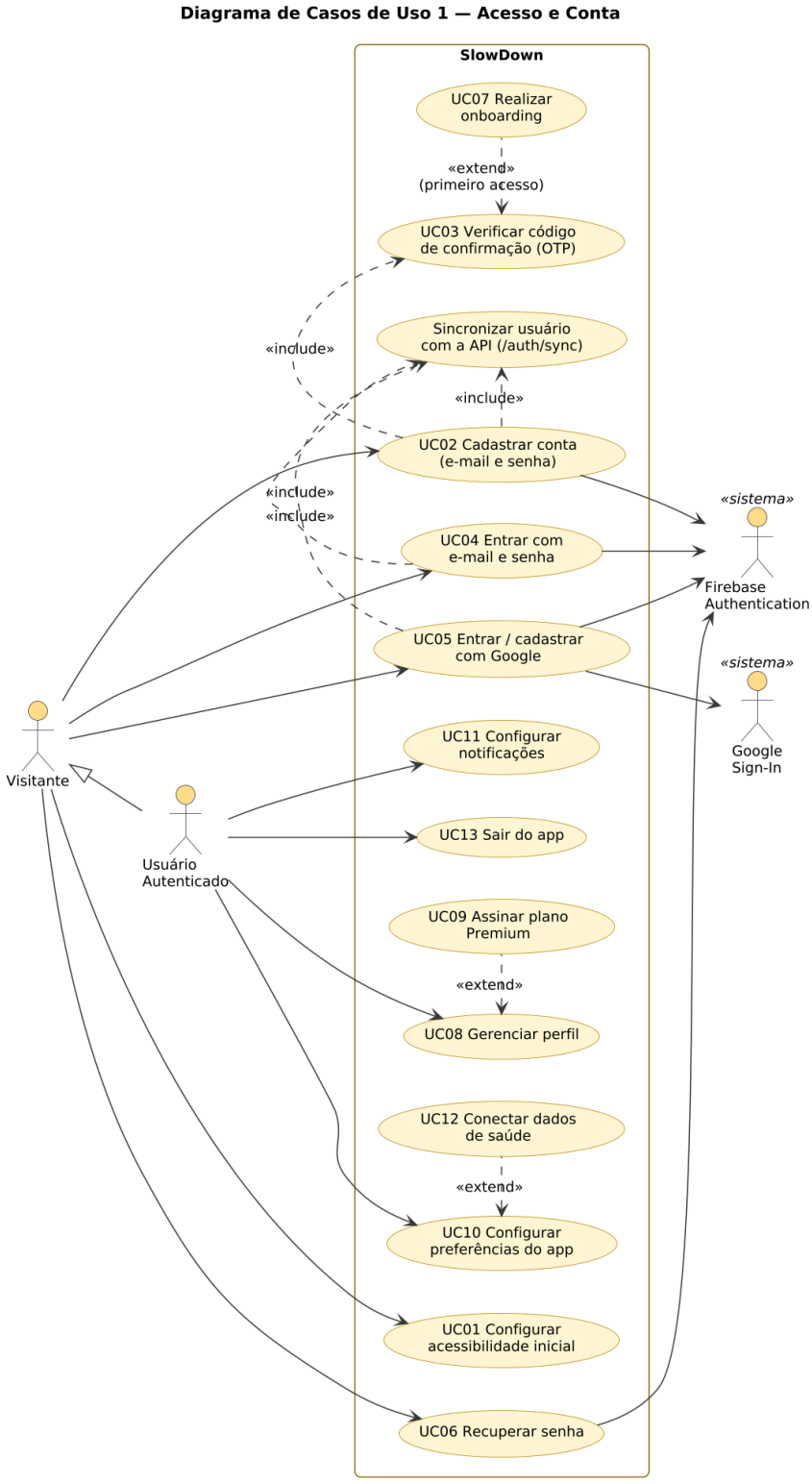


Figura 1 — Casos de uso de acesso e conta.

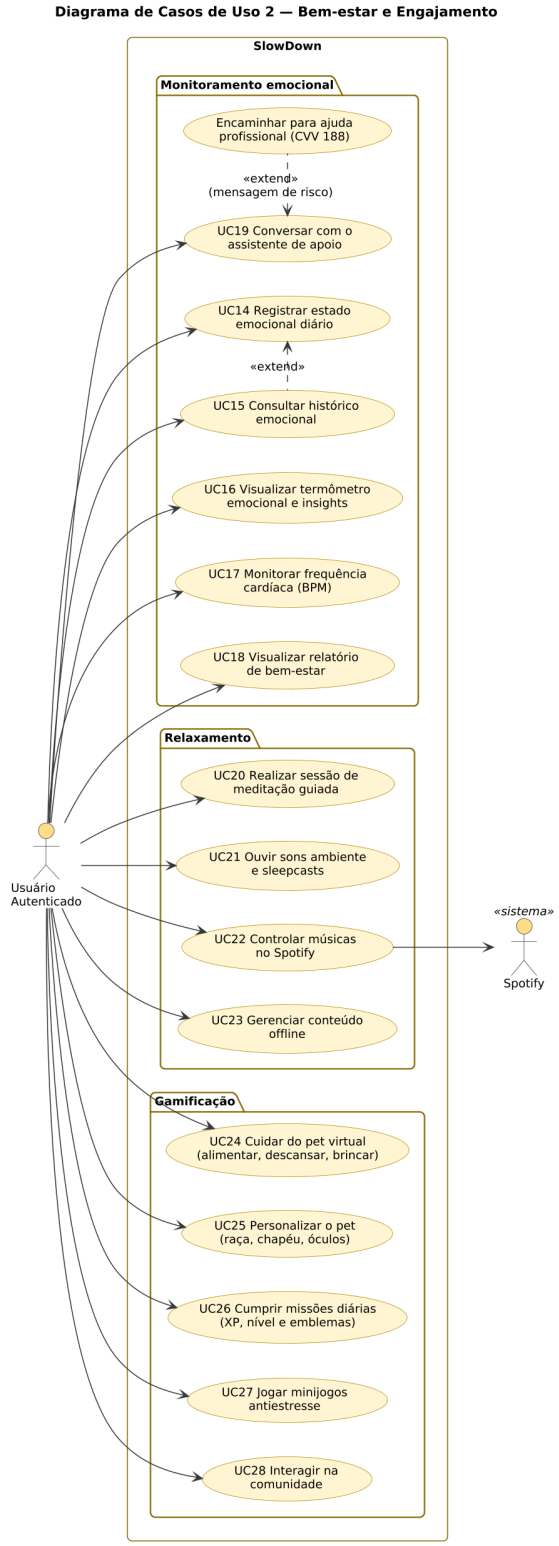


Figura 2 — Casos de uso de bem-estar e engajamento.

Rastreabilidade casos de uso × requisitos

UC	Caso de uso	RF	UC	Caso de uso	RF
UC01	Configurar acessibilidade inicial	RF01	UC15	Consultar histórico emocional	RF11
UC02	Cadastrar conta (e-mail e senha)	RF02, RF07	UC16	Visualizar termômetro e insights	RF12
UC03	Verificar código de confirmação	RF03	UC17	Monitorar frequência cardíaca	RF13
UC04	Entrar com e-mail e senha	RF04, RF07, RF08	UC18	Visualizar relatório	RF14
UC05	Entrar / cadastrar com Google	RF05, RF07	UC19	Conversar com o assistente	RF15
UC06	Recuperar senha	RF06	UC20	Realizar meditação guiada	RF16
UC07	Realizar onboarding	RF09	UC21	Ouvir sons e sleepcasts	RF17
UC08	Gerenciar perfil	RF27	UC22	Controlar músicas no Spotify	RF18
UC09	Assinar plano Premium	RF26	UC23	Gerenciar conteúdo offline	RF19
UC10	Configurar preferências	RF28	UC24	Cuidar do pet virtual	RF20
UC11	Configurar notificações	RF25	UC25	Personalizar o pet	RF21
UC12	Conectar dados de saúde	RF29	UC26	Cumprir missões diárias	RF22, RF23
UC13	Sair do app	RF08	UC27	Jogar minijogos	RF24
UC14	Registrar estado emocional	RF10	UC28	Interagir na comunidade	RF30

3.2. Especificação dos Casos de Uso

A seguir, a especificação completa dos casos de uso principais. Os demais aparecem de forma resumida ao final da seção.

UC02 — Cadastrar conta (e-mail e senha)

Atores	Visitante; Firebase Authentication
Pré-condições	App aberto na tela de cadastro.
Pós-condições	Conta criada no Firebase e registro criado na tabela `users`; sessão salva no aparelho; usuário na tela de verificação.
Fluxo principal	1. O usuário informa nome, e-mail, senha e confirmação e marca o aceite dos Termos. 2. O usuário toca **CRIAR CONTA**. 3. O sistema valida nome, e-mail, senha e confirmação (A1). 4. O sistema verifica o aceite dos Termos (A2). 5. O sistema cria a conta no Firebase e define o nome de exibição (A3). 6. O sistema obtém o token e chama `POST /auth/sync`; a API cria o usuário com plano padrão. 7. O sistema salva a sessão com o id do MySQL e exibe “Quase lá! Verifique seu e-mail.”. 8. O sistema abre a verificação de código (UC03).
Fluxos alternativos / exceção	• A1 — Dado inválido: exibe a primeira mensagem de erro (ex.: “A senha deve conter pelo menos um número.”) e permanece na tela. • A2 — Termos não aceitos: exibe “Aceite os termos para continuar.”. • A3 — E-mail já usado: exibe “Este e-mail já está em uso.”. • A4 — Falha de rede ou da API: exibe a mensagem de erro retornada.
Regras de negócio	RN01, RN02, RN05
Implementação	`register_screen.dart`, `auth_provider.dart`, `authController.syncUser`

UC03 — Verificar código de confirmação (OTP)

Atores	Visitante recém-cadastrado
Pré-condições	Cadastro por e-mail concluído (UC02).
Pós-condições	Conta confirmada; usuário no onboarding (primeiro acesso) ou na Home.
Fluxo principal	1. O sistema exibe o campo do código e a contagem regressiva de 15:00. 2. O usuário digita os 6 dígitos e confirma. 3. O sistema valida formato e prazo localmente. 4. O sistema envia `POST /auth/verify-otp`. 5. A API confirma o código. 6. Se for o primeiro acesso, abre o onboarding (UC07); senão, a Home.
Fluxos alternativos / exceção	• A1 — Menos de 6 dígitos: “O código deve ter 6 dígitos.”. • A2 — Código diferente do esperado: “Código de verificação inválido.”. • A3 — Prazo esgotado: “Código expirado. Solicite um novo código.”.
Regras de negócio	RN04
Implementação	`otp_verification_screen.dart`, `AuthService.verificarCodigo()`

UC04 — Entrar com e-mail e senha

Atores	Visitante; Firebase Authentication
Pré-condições	Usuário possui conta.
Pós-condições	Sessão salva (token, id, nome) e Home aberta.
Fluxo principal	1. O usuário informa e-mail e senha e toca **ENTRAR**. 2. O sistema verifica se os campos estão preenchidos (A1). 3. O sistema autentica no Firebase (A2). 4. O sistema salva o token e sincroniza com `POST /auth/sync`. 5. A API devolve o usuário existente; o sistema atualiza a sessão com o id do MySQL. 6. O sistema abre a Home.
Fluxos alternativos / exceção	• A1 — Campo vazio: “Por favor, preencha e-mail e senha.”. • A2 — Credenciais inválidas: “E-mail ou senha incorretos.”. • A3 — O usuário escolhe “Esqueci a senha” (UC06) ou “Criar minha conta” (UC02).
Regras de negócio	RN03, RN05, RN06
Implementação	`login_screen.dart`, `AuthNotifier.login()`

UC05 — Entrar / cadastrar com Google

Atores	Visitante; Google Sign-In; Firebase Authentication
Pré-condições	Conta Google disponível no aparelho.
Pós-condições	Usuário autenticado, sincronizado no MySQL e na Home (sem OTP).
Fluxo principal	1. O usuário toca “Continuar com Google” (login) ou “Criar com Google” (cadastro). 2. O sistema abre o seletor de contas do Google. 3. O usuário escolhe a conta (A1). 4. O sistema autentica no Firebase com a credencial Google. 5. O sistema chama `POST /auth/sync` enviando o nome da conta; a API cria (201) ou devolve (200) o usuário. 6. O sistema salva a sessão e abre a Home.
Fluxos alternativos / exceção	• A1 — Seleção cancelada: “Login com Google cancelado.”. • A2 — Falha de rede: “Não foi possível conectar ao Google. Verifique sua conexão.”.
Regras de negócio	RN04, RN05
Implementação	`AuthService.loginComGoogle()`

UC06 — Recuperar senha

Atores	Visitante; Firebase Authentication
Pré-condições	Usuário na tela de login.
Pós-condições	E-mail de redefinição enviado pelo Firebase.
Fluxo principal	1. O usuário toca “Esqueci a Senha”. 2. O usuário informa o e-mail e solicita o link. 3. O sistema valida o e-mail. 4. O sistema chama `sendPasswordResetEmail()`. 5. O sistema exibe a confirmação de envio.
Fluxos alternativos / exceção	• A1 — E-mail inválido: exibe o erro de validação. • A2 — E-mail não cadastrado: exibe aviso correspondente. • A3 — Sem conexão: “Não foi possível conectar ao servidor. Verifique sua conexão.”.
Regras de negócio	RN02
Implementação	`forgot_password_screen.dart`

UC14 — Registrar estado emocional diário

Atores	Usuário autenticado
Pré-condições	Usuário na tela Saúde.
Pós-condições	Novo `EmotionalEntry` salvo no aparelho e visível no histórico e no termômetro.
Fluxo principal	1. O usuário abre **Registro emocional**. 2. Ajusta a escala de 1 a 10 e, se quiser, escolhe emoji, cor e escreve uma nota. 3. Toca **SALVAR**. 4. O sistema valida escala, emoji e cor (A1) e a nota (A2). 5. O sistema grava o registro com data e hora atuais. 6. O sistema mostra a confirmação e o atalho para o histórico.
Fluxos alternativos / exceção	• A1 — Emoji ou cor fora do conjunto: exibe o primeiro erro. • A2 — Nota acima de 500 caracteres: “A nota não pode ultrapassar 500 caracteres.”. • A3 — Sem emoji/cor: o registro é salvo com emoji vazio e cor neutra `#808080`.
Regras de negócio	RN07
Implementação	`emotional_record_screen.dart`, `ValidadorRegistroEmocional`, `EmotionalEntryStorage`

UC17 — Monitorar frequência cardíaca

Atores	Usuário autenticado
Pré-condições	Usuário na Home.
Pós-condições	Leitura exibida e classificada.
Fluxo principal	1. O usuário toca **Saúde**. 2. O sistema mostra o BPM atual, sua classificação e o histórico da semana. 3. O usuário toca **START**. 4. O sistema inicia a animação de pulso e obtém uma nova leitura. 5. O sistema classifica a leitura e atualiza cor e rótulo. 6. O usuário toca **PARAR** para encerrar.
Fluxos alternativos / exceção	• A1 — Leitura < 60 ou > 100 BPM: exibe “ABAIXO DO NORMAL” ou “ELEVADO”. • A2 — O usuário segue para registro emocional, assistente, histórico, termômetro ou relatório.
Regras de negócio	RN08
Implementação	`monitor_screen.dart`, `ValidadorBpm`. No MVP a leitura é simulada (72–87 BPM).

UC19 — Conversar com o assistente de apoio

Atores	Usuário autenticado
Pré-condições	Usuário na tela Saúde.
Pós-condições	Conversa salva no aparelho.
Fluxo principal	1. O usuário abre o assistente; o sistema carrega histórico e estilo salvos. 2. O usuário escreve e envia uma mensagem. 3. O sistema verifica se a mensagem indica risco (A1). 4. O sistema gera a resposta conforme o estilo escolhido (acolhedor, direto ou motivacional). 5. O sistema exibe a resposta e salva o histórico.
Fluxos alternativos / exceção	• A1 — Mensagem de risco: exibe primeiro o cartão de ajuda com o CVV 188. • A2 — O usuário troca o estilo do assistente ou limpa o histórico.
Regras de negócio	RN13
Implementação	`chatbot_screen.dart` (`_LocalChatEngine`, `ChatStorage`)

UC20 — Realizar sessão de meditação guiada

Atores	Usuário autenticado
Pré-condições	Usuário na Home.
Pós-condições	Sessão concluída.
Fluxo principal	1. O usuário toca **Meditação** e lê a introdução. 2. Toca **COMEÇAR A MEDITAR** e escolhe uma sessão (Foco Mental 10 min, Sono Profundo 15 min, Reduzir Ansiedade 8 min ou Energia Matinal 5 min). 3. O sistema exibe o temporizador com a duração da sessão e a animação de respiração. 4. O usuário inicia; o sistema conta o tempo e alterna as instruções de respiração. 5. Ao zerar, o sistema indica a conclusão.
Fluxos alternativos / exceção	• A1 — O usuário pausa e retoma a sessão. • A2 — O usuário sai antes do fim; o temporizador é cancelado.
Regras de negócio	—
Implementação	`about_meditation_screen.dart`, `meditation_screen.dart`, `meditation_session_screen.dart`

UC24 — Cuidar do pet virtual

Atores	Usuário autenticado
Pré-condições	Usuário na Home.
Pós-condições	Status e aparência do pet salvos no aparelho.
Fluxo principal	1. O usuário toca **Pet**; o sistema carrega status, raça e acessórios. 2. O sistema reduz os status a cada 30 s enquanto a tela está aberta. 3. O usuário escolhe **Alimentar**, **Descansar** ou **START** (brincar). 4. O sistema aplica o efeito da ação, salva e atualiza a expressão do pet. 5. Ao terminar o minijogo do pet, o sistema soma felicidade, desconta energia e fome e mostra fogos de artifício.
Fluxos alternativos / exceção	• A1 — Fome em 100: “Seu pet já está cheio!”. • A2 — Energia em 100: “Seu pet está cheio de energia!”. • A3 — Energia < 10 ao brincar: “Seu pet está cansado!”. • A4 — O usuário toca **SKINS** e personaliza o pet (UC25).
Regras de negócio	RN11
Implementação	`pet_screen.dart`, `pet_skin_screen.dart`

UC26 — Cumprir missões diárias

Atores	Usuário autenticado
Pré-condições	Usuário no Menu.
Pós-condições	XP, nível, sequência e emblemas atualizados e salvos.
Fluxo principal	1. O usuário abre **Missões**; o sistema carrega o progresso e reinicia as missões se mudou o dia. 2. O sistema mostra nível, barra de XP, sequência e as 4 missões do dia. 3. O usuário marca uma missão como feita. 4. O sistema soma o XP, atualiza a sequência e o nível e verifica os emblemas. 5. O sistema salva o progresso e destaca emblemas novos.
Fluxos alternativos / exceção	• A1 — Missão já concluída hoje: nada é somado.
Regras de negócio	RN09, RN10
Implementação	`missions_screen.dart` (`GamificationStatus`, `kBadgeCatalog`)

UC09 — Assinar plano Premium

Atores	Usuário autenticado
Pré-condições	Usuário no Perfil.
Pós-condições	Status Premium ativo com data de expiração.
Fluxo principal	1. O usuário toca no botão Premium do perfil. 2. O sistema lista benefícios e os planos mensal e anual. 3. O usuário escolhe um plano e confirma. 4. O sistema ativa o Premium, calcula a expiração e salva.
Fluxos alternativos / exceção	• A1 — O usuário cancela a confirmação. • A2 — Usuário Premium cancela a assinatura: o status volta a não Premium.
Regras de negócio	RN12
Implementação	`premium_screen.dart` (`PremiumStatus`). No MVP não há cobrança real.

Especificação resumida dos demais casos de uso

UC	Descrição do fluxo	Implementação
UC01	Na abertura (após a Splash), o visitante liga ou desliga os comandos de voz e segue para o login.	`accessibility_screen.dart`
UC07	No primeiro acesso após a verificação, o usuário vê os slides de apresentação e escolhe apelido e objetivo (reduzir ansiedade, dormir melhor, melhorar o foco ou entender o humor).	`onboarding_screen.dart`, `profile_setup_screen.dart`
UC08	Menu → Perfil: exibe dados do usuário e o acesso ao Premium.	`profile_screen.dart`
UC10	Menu → Configurações: idioma, modo silencioso, permissões, qualidade de mídia e acessibilidade.	`settings_screen.dart`
UC11	Menu → Notificações: liga/desliga, permissão do sistema, horário do lembrete e categorias (RN14).	`notifications_screen.dart`
UC12	Configurações → HealthKit & Google Fit: conectar/desconectar provedores e ver a última sincronização.	`health_sync_screen.dart`
UC13	Menu → Sair: encerra a sessão do Firebase, apaga o token local e volta ao login.	`menu_screen.dart`

UC	Descrição do fluxo	Implementação
UC15	Lista os registros (mais recente primeiro) com média, melhor e pior e permite limpar.	`emotional_history_screen.dart`
UC16	Mostra o termômetro do dia, barras dos últimos 7 dias, linha de 30 dias, tendência semanal e sugestões.	`emotional_thermometer_screen.dart`
UC18	Filtra semana ou mês e mostra estatísticas de humor, meditação e BPM.	`report_screen.dart`
UC21	Aba Sons ambiente (acústico, chuva, natureza, ruído branco) com mini player; acesso aos sleepcasts com temporizador.	`music_screen.dart`, `sleepcasts_screen.dart`
UC22	Aba Músicas → Abrir Spotify: autentica no app Spotify e controla playlists e faixas.	`spotify_screen.dart`
UC23	Menu → Offline: lista itens salvos, espaço ocupado, opções Wi-Fi e sincronização automática; remover item ou tudo (RN15).	`offline_screen.dart`
UC25	Escolher raça, chapéu e óculos com pré-visualização e confirmar.	`pet_skin_screen.dart`
UC27	Menu → Jogos: Quiz (9 perguntas), Corrida (3 faixas, obstáculos e moedas) e Damas contra o computador.	`games_screen.dart` e jogos
UC28	Home → Comunidade: perfil com avatar, feed para compartilhar humor e mensagens.	`social_screen.dart`

3.3. Diagramas de Sequência

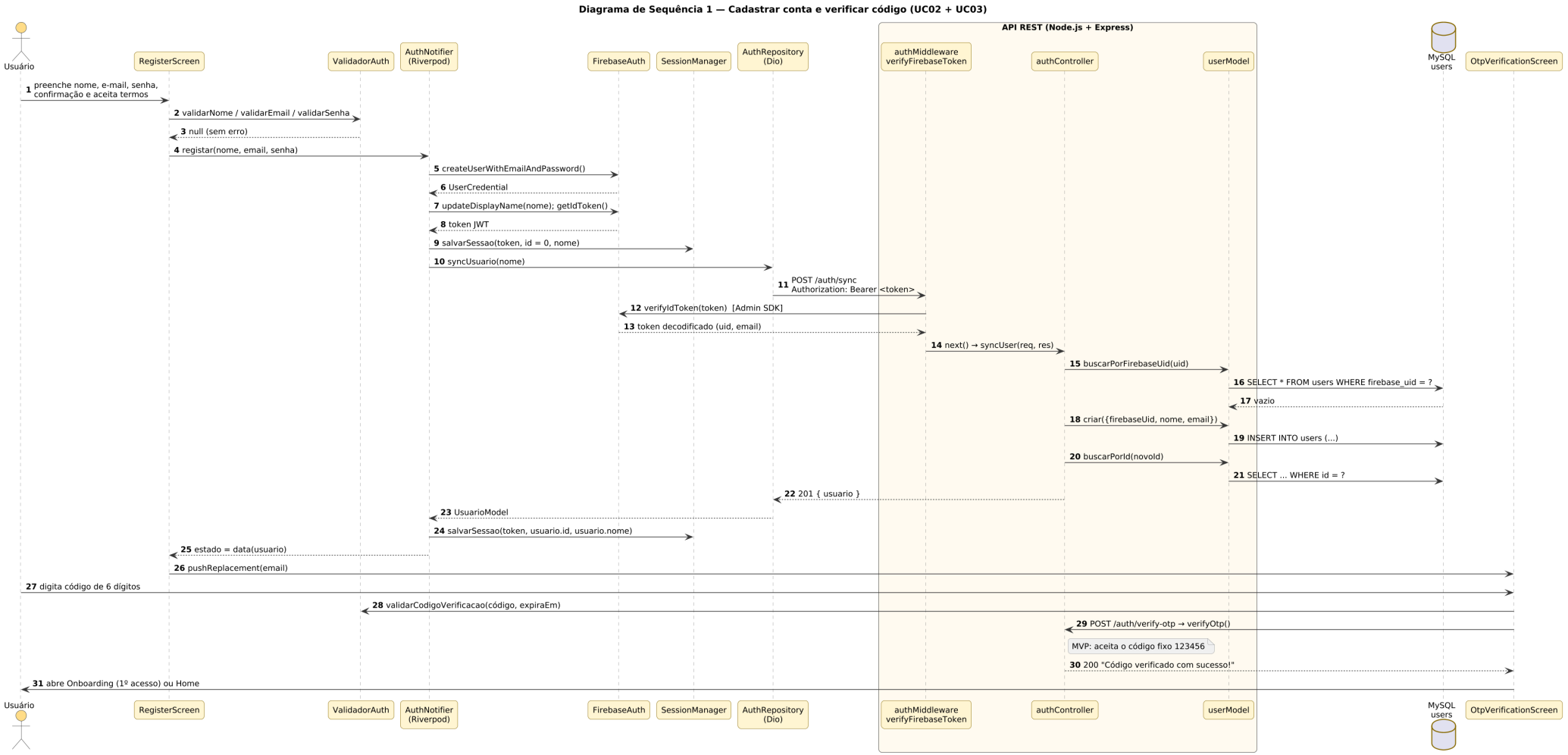


Figura 3 — Cadastro com e-mail e senha e verificação do código (UC02 e UC03). Atravessa as três camadas: View (Flutter), Controller (Express) e Model (MySQL).

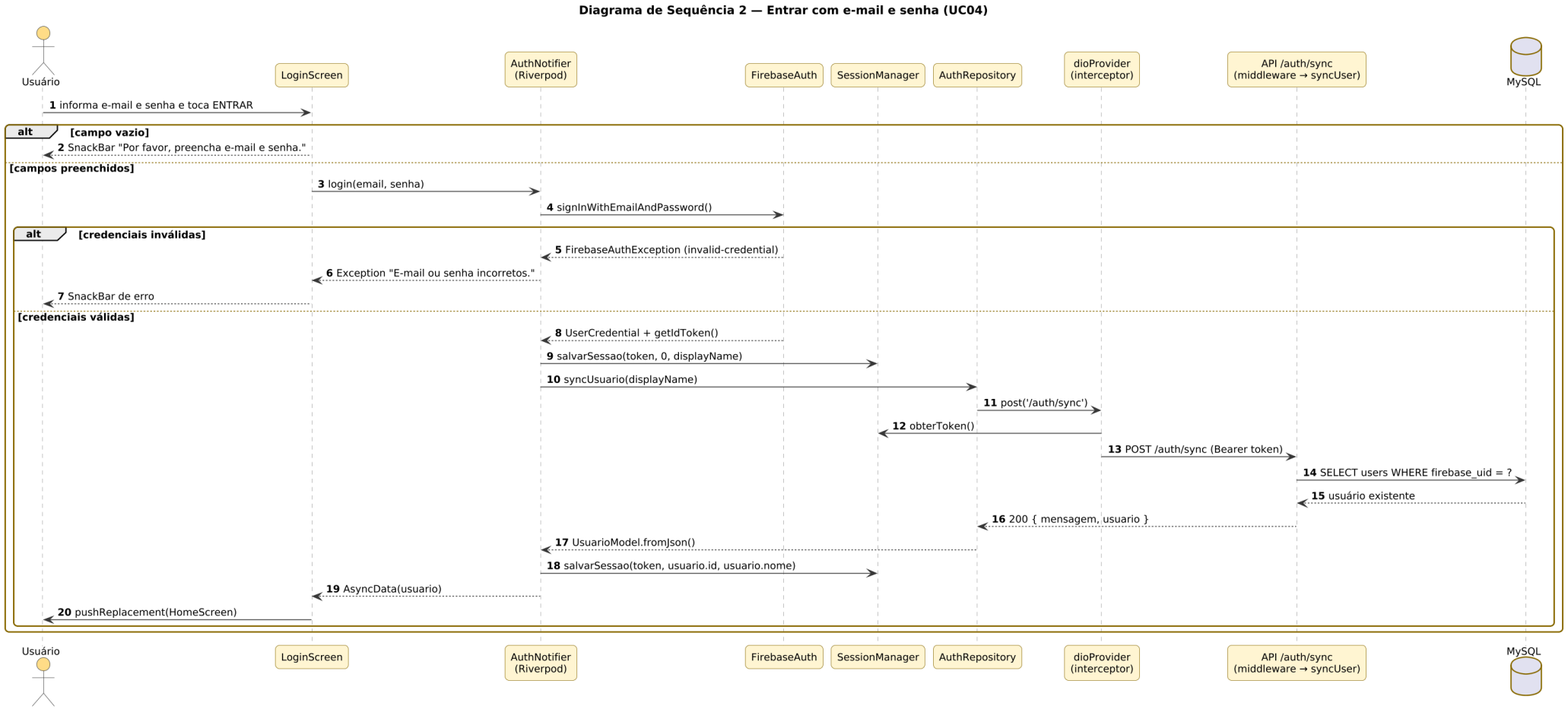


Figura 4 — Login com e-mail e senha (UC04): o interceptor do Dio injeta o token em toda chamada à API.

Diagrama de Sequência 3 — Entrar ou cadastrar com Google (UC05)

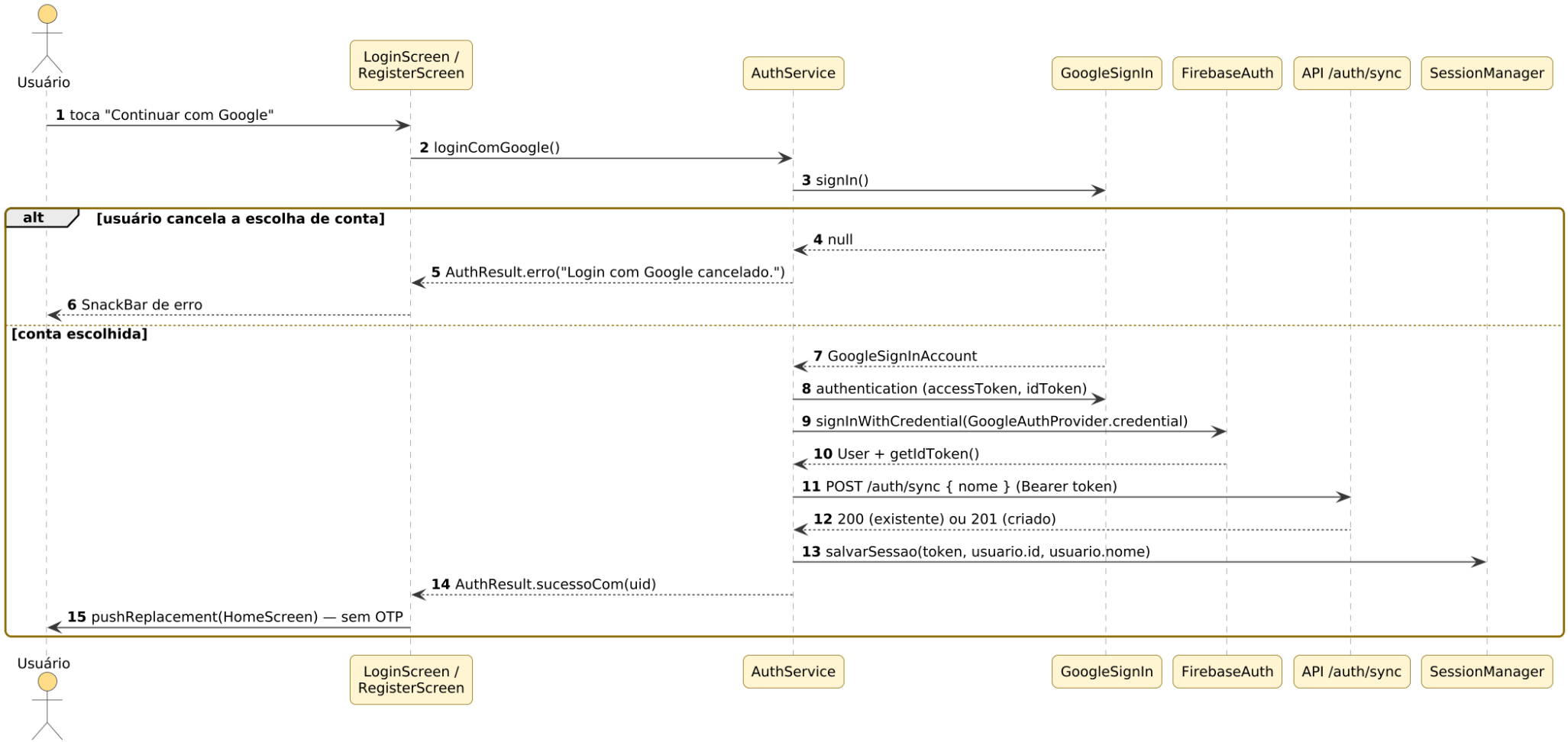


Figura 5 — Login ou cadastro com Google (UC05).

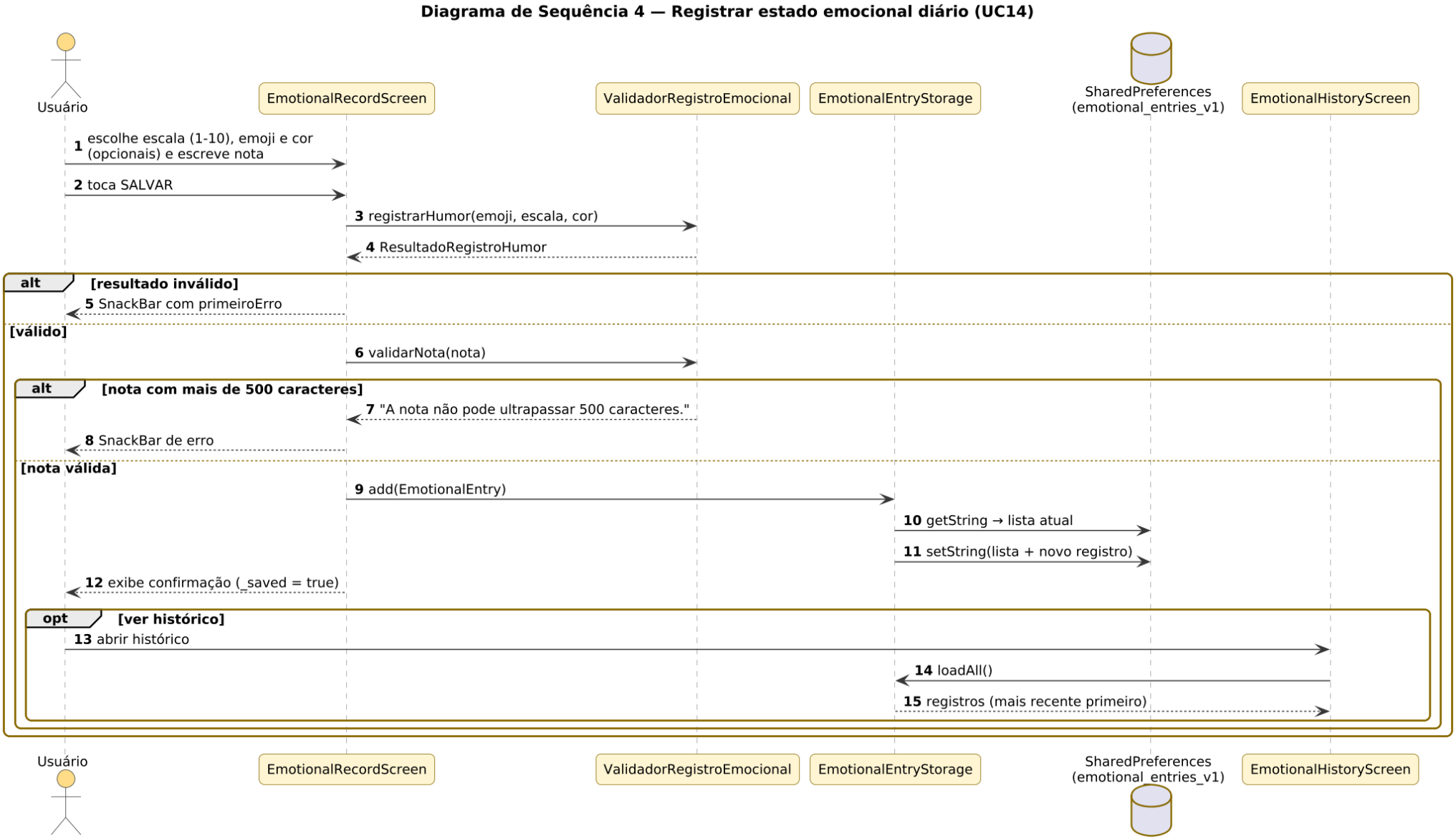


Figura 6 — Registro emocional (UC14): persistência local via EmotionalEntryStorage.

Diagrama de Sequência 5 — Conversar com o assistente de apoio (UC19)

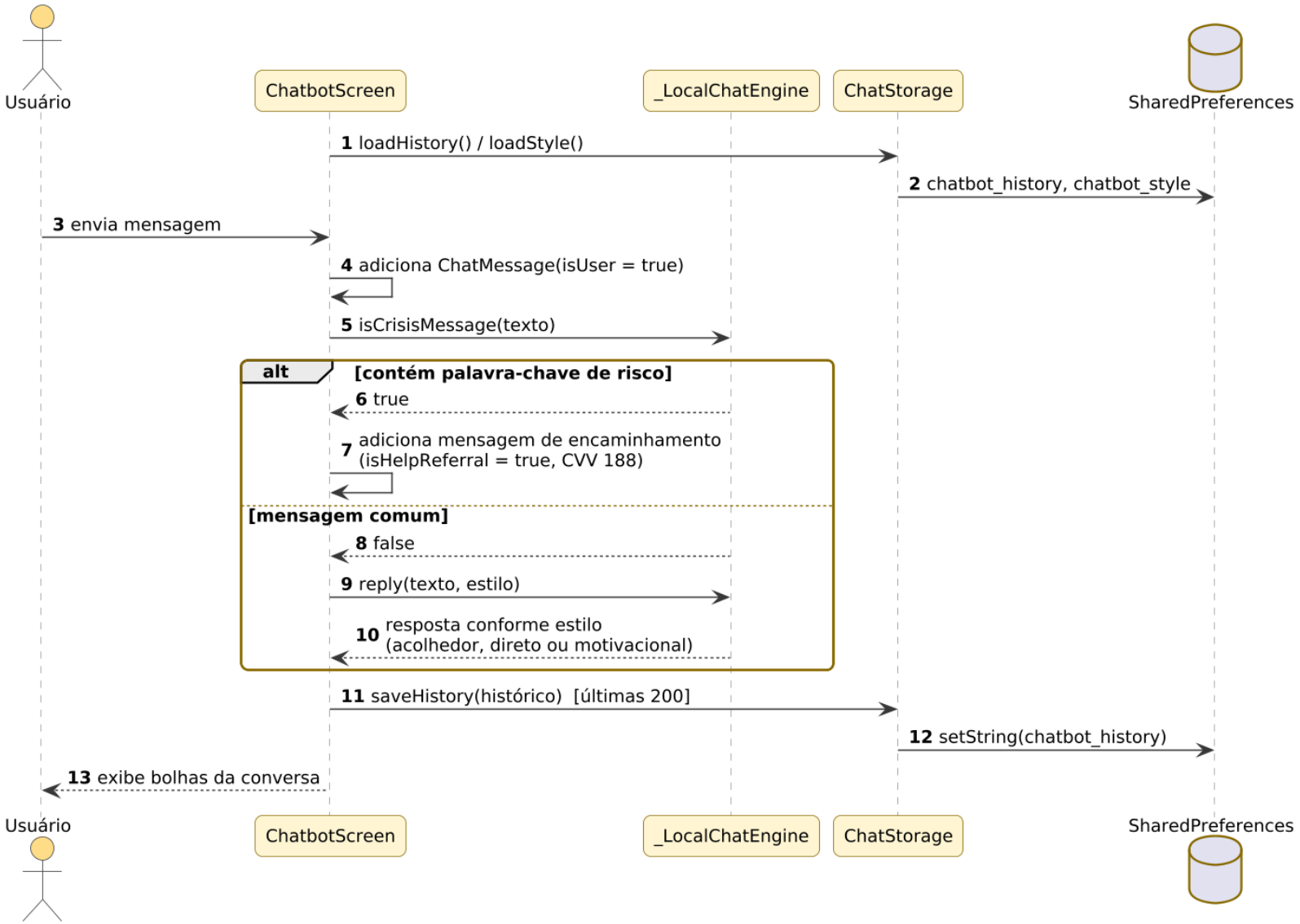


Figura 7 — Conversa com o assistente (UC19), incluindo a detecção de mensagem de risco.

3.4. Diagramas de Atividades

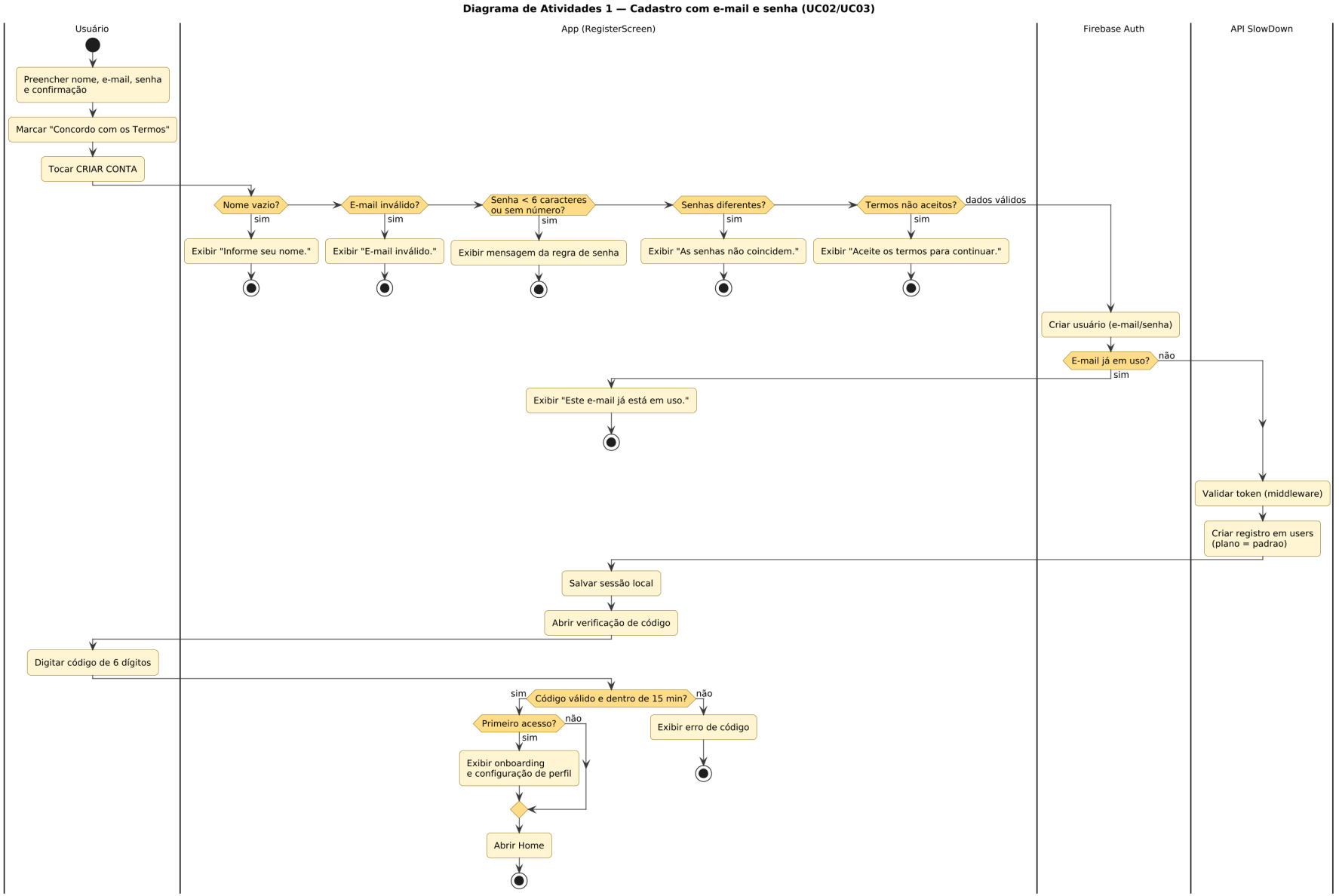


Figura 8 — Cadastro com e-mail e senha, com raias Usuário, App, Firebase e API.

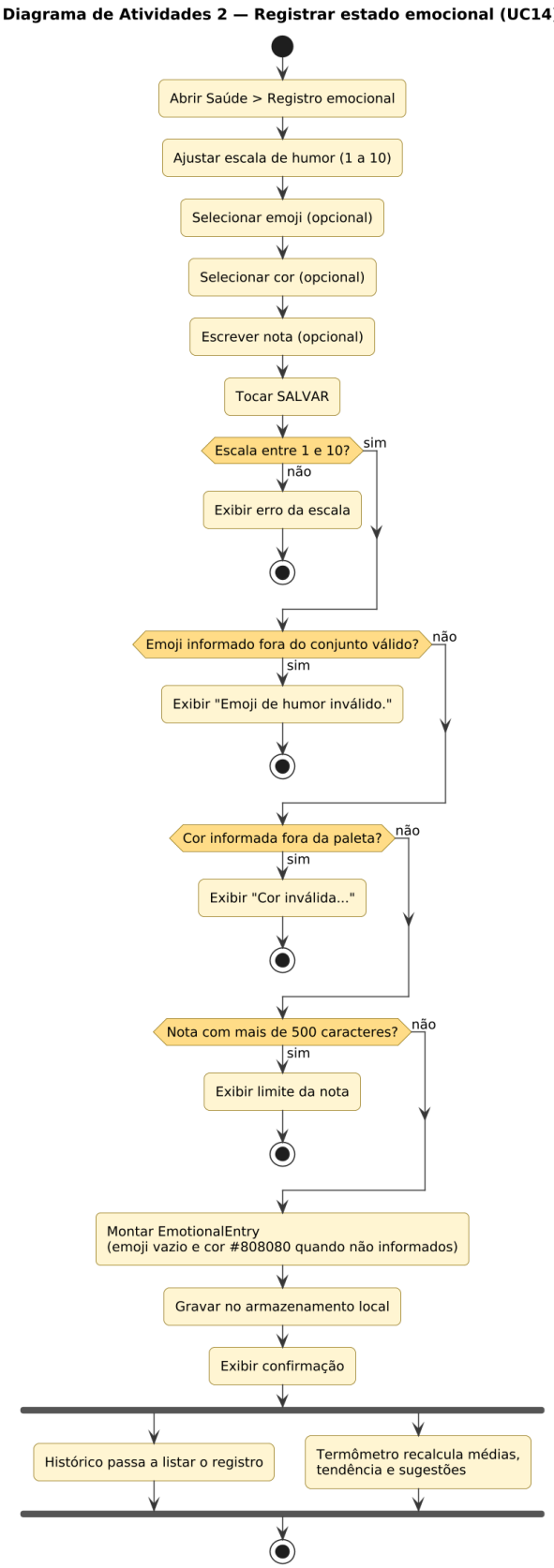


Figura 9 — Registrar estado emocional (UC14).

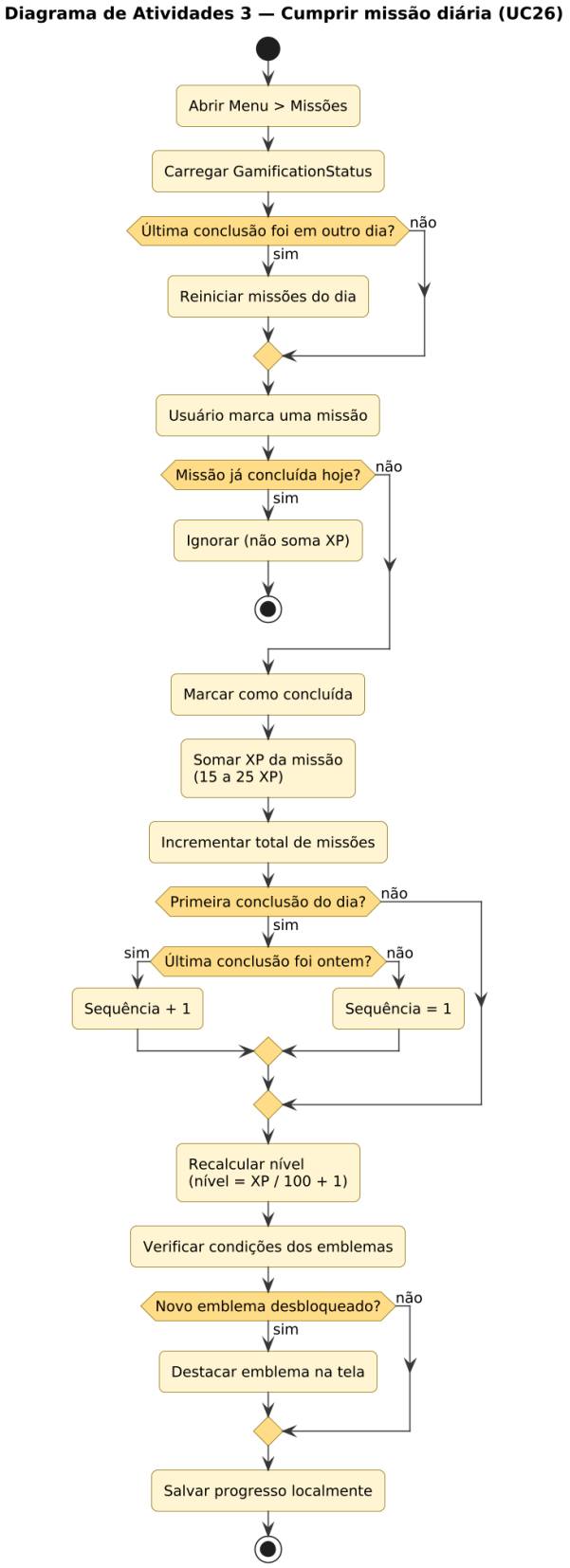


Figura 10 — Cumprir missão diária (UC26).

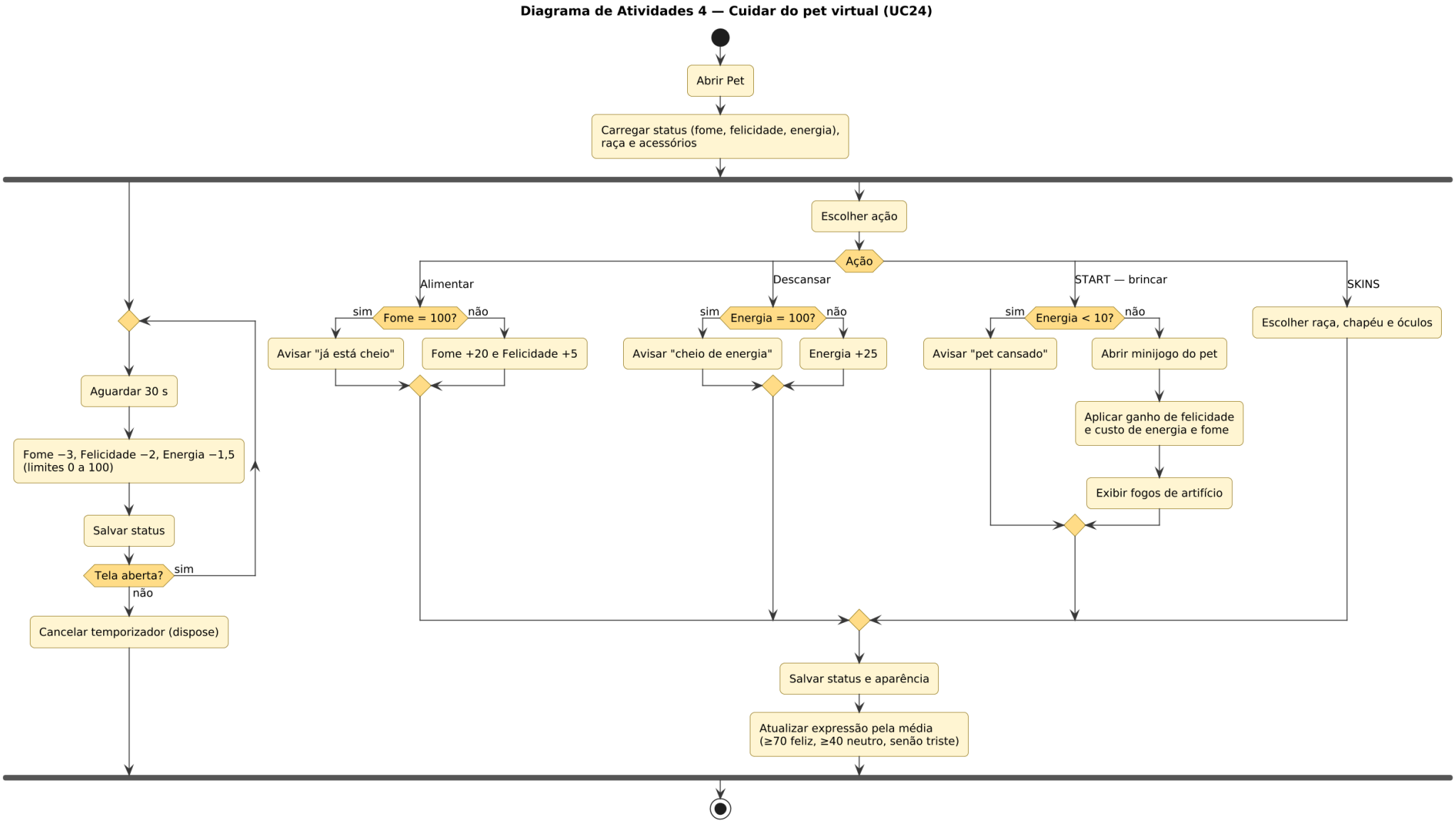


Figura 11 — Cuidar do pet virtual (UC24): o decaimento dos status corre em paralelo às ações do usuário.

Diagrama de Atividades 5 — Monitorar frequência cardíaca (UC17)

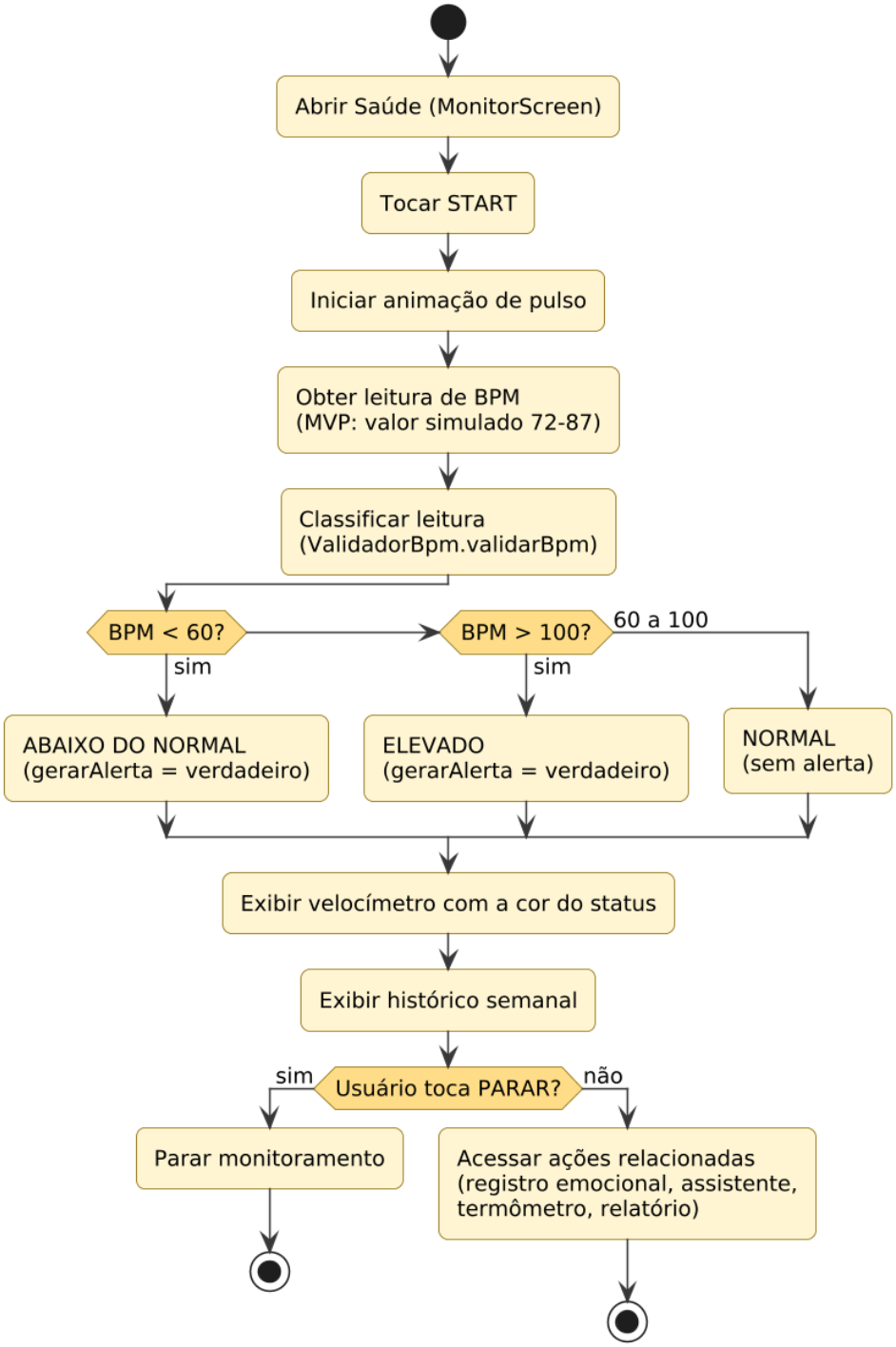


Figura 12 — Monitorar frequência cardíaca (UC17).

3.5. Diagrama de Classes

O diagrama de classes foi dividido em quatro vistas, todas extraídas do código: (1) a API REST em MVC com a tabela `users`; (2) o núcleo do app — telas de autenticação, provider Riverpod, repositório, serviços e validadores; (3) e (4) os modelos de domínio que o app persiste localmente.

Diagrama de Classes 1 — API REST (Node.js + Express), padrão MVC

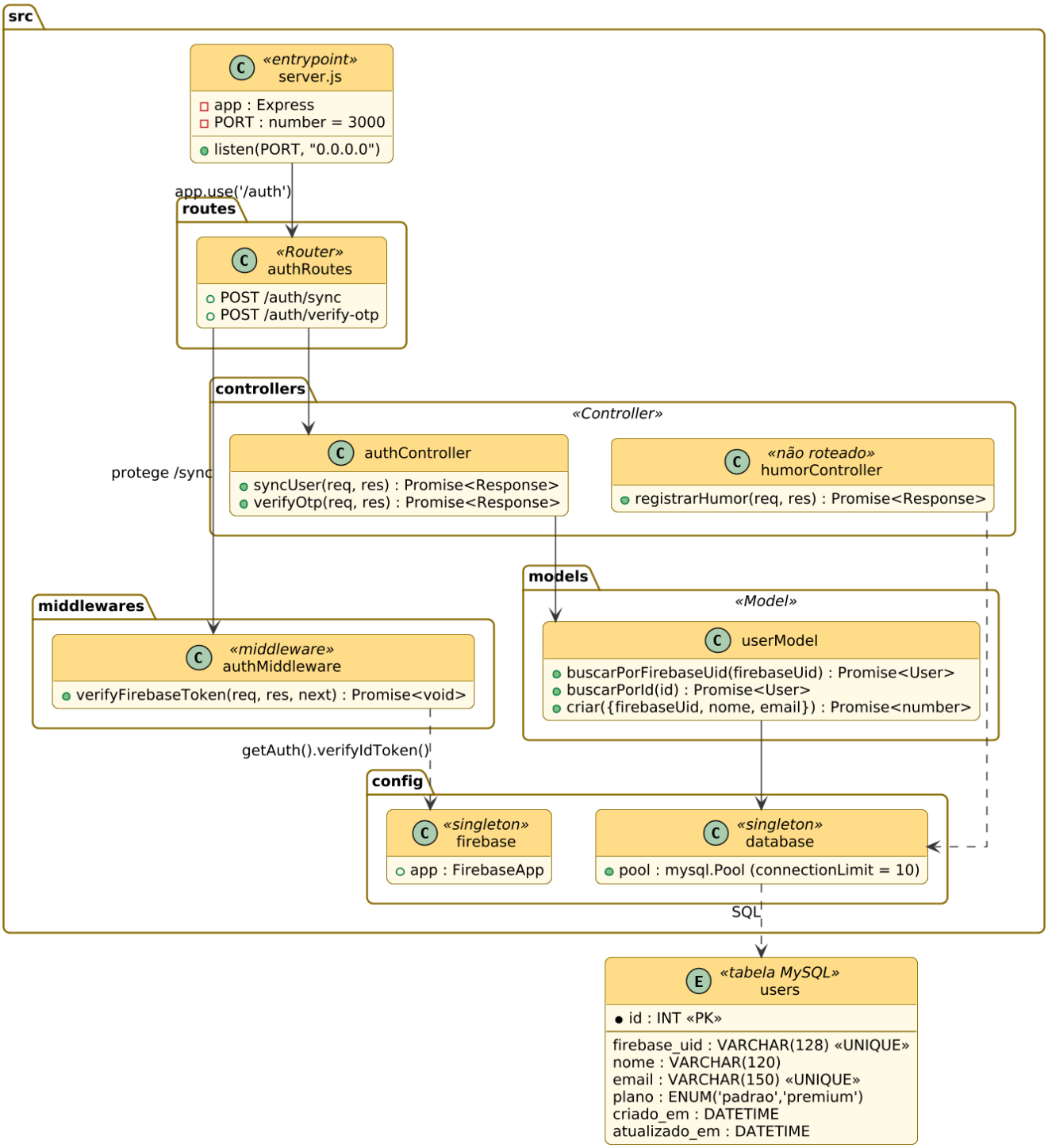


Figura 13 — Classes da API REST (Node.js + Express) e tabela users do MySQL.

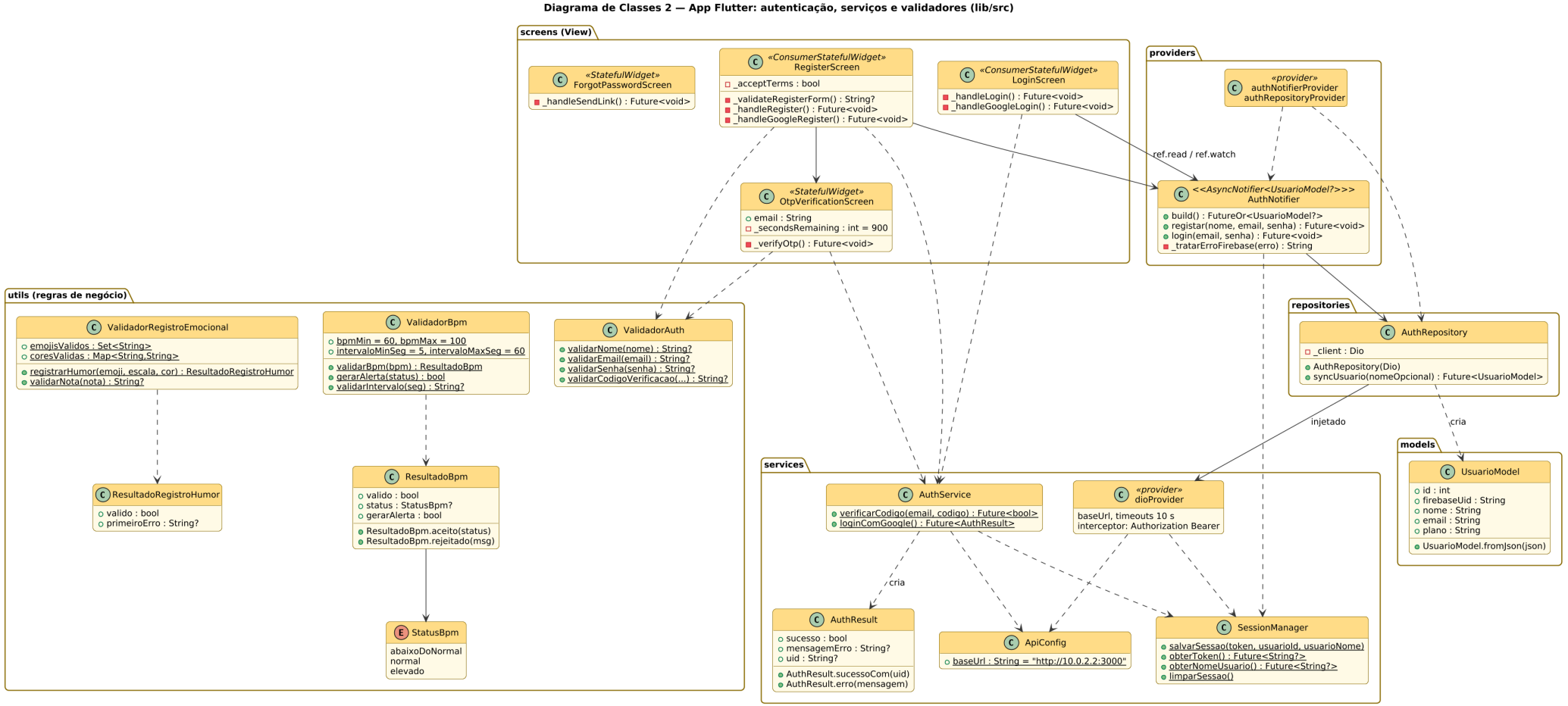


Figura 14 — App Flutter: autenticação, estado (Riverpod), serviços e regras de negócio (lib/src).

Diagrama de Classes 3 — App Flutter: modelos de domínio persistidos localmente (lib/screens) — parte A

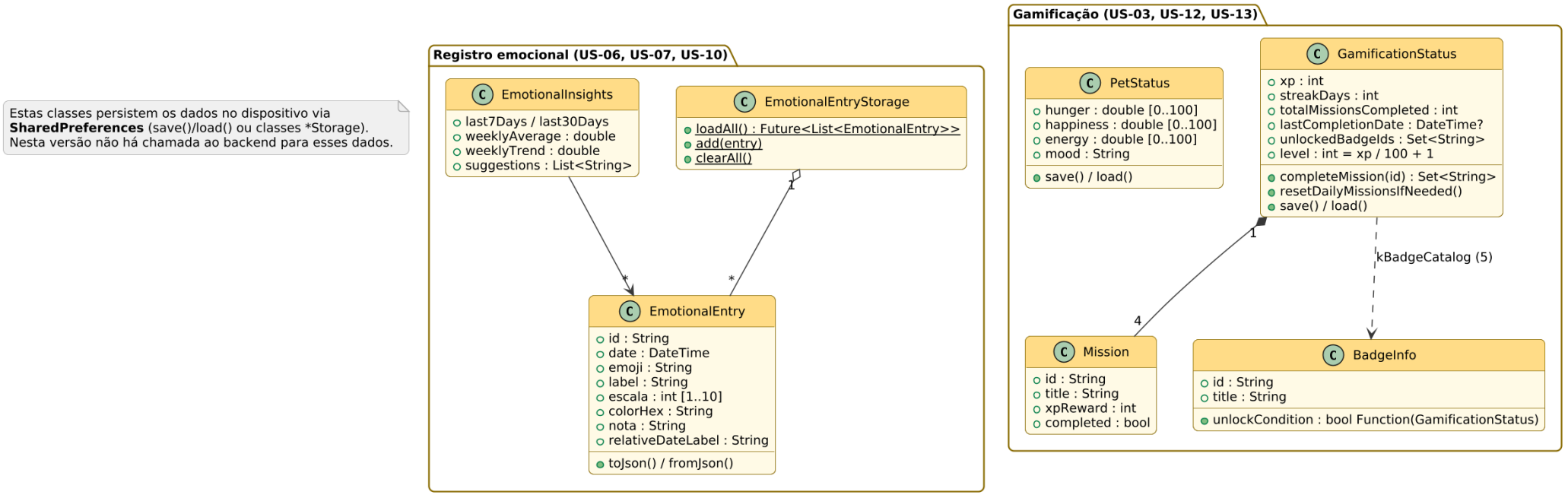


Figura 15 — Modelos de domínio: registro emocional e gamificação.

Diagrama de Classes 4 — App Flutter: modelos de domínio persistidos localmente (lib/screens) — parte B

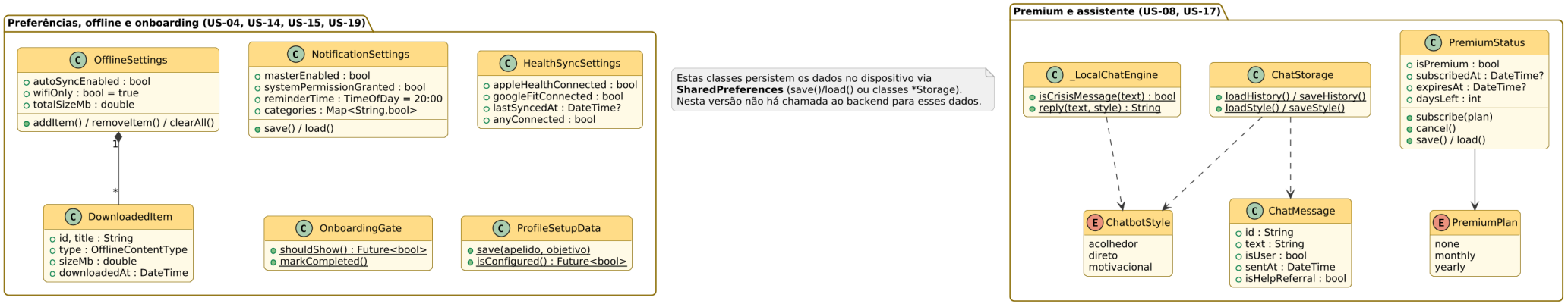


Figura 16 — Modelos de domínio: Premium, assistente, preferências, offline e onboarding.

4. Arquitetura

O SlowDown segue o estilo **cliente-servidor**, com o padrão arquitetural **MVC (Model-View-Controller)** definido no TP2: a **View** é o app Flutter, o **Controller** é a API REST em Express e o **Model** é a camada que acessa o MySQL. A autenticação é delegada a um serviço externo (Firebase). A Figura 17 mostra os containers efetivamente implementados.

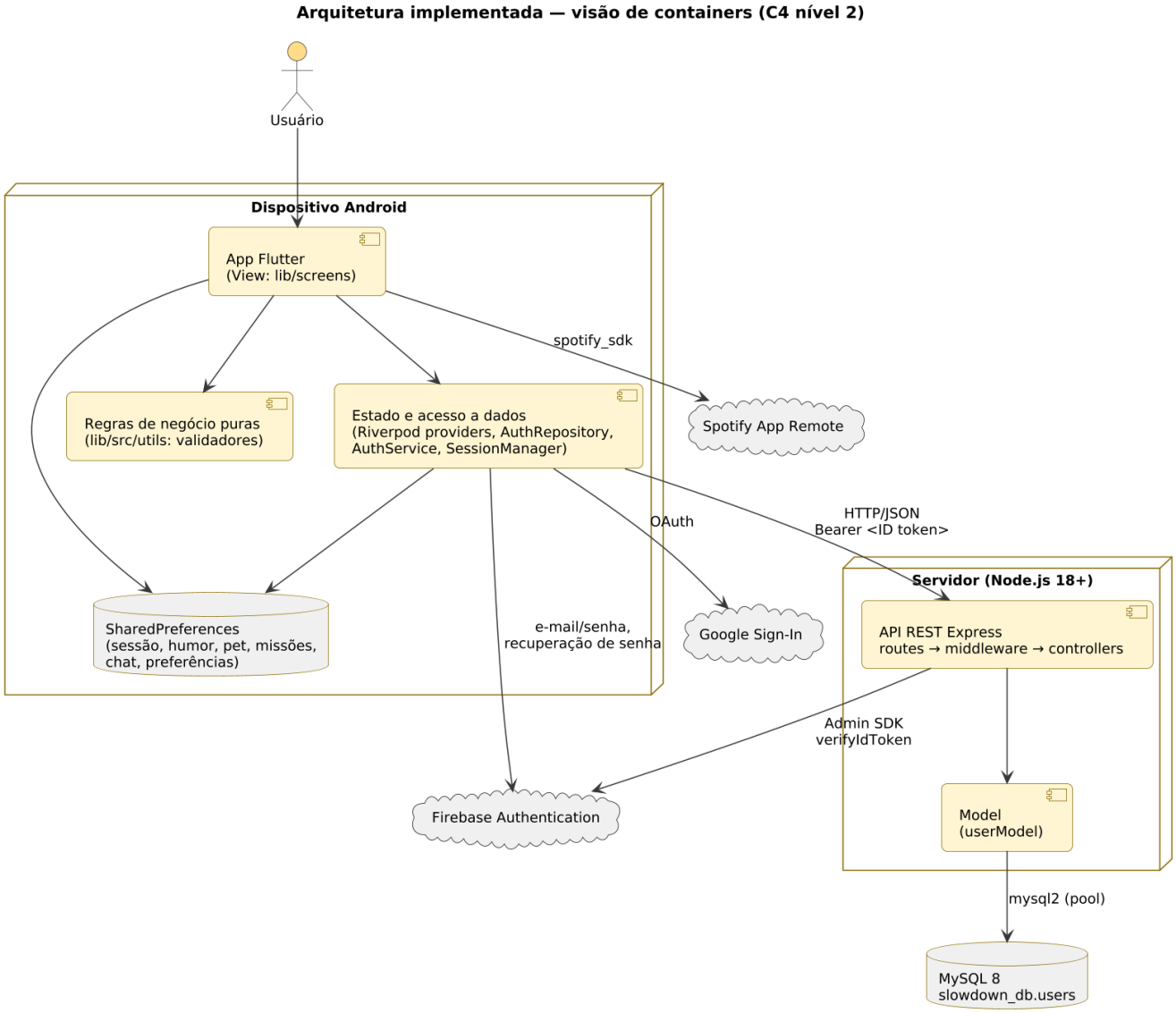


Figura 17 — Visão de containers do sistema implementado.

Camadas e responsabilidades

Camada MVC	Onde está no código	Responsabilidade
View	`frontend/lib/screens/` (42 telas), `splash_screen.dart`, `accessibility_screen.dart`	Interface, captura de ações e navegação. Não executa SQL nem conhece a URL da API diretamente.
Apoio à View (cliente)	`lib/src/providers`, `repositories`, `services`, `models`, `utils`	Estado da autenticação (Riverpod), chamadas HTTP (Dio), sessão local, DTO do usuário e validadores das regras de negócio.
Controller	`backend/src/routes`, `middlewares`, `controllers`	Recebe a requisição, valida o token e aplica a regra (criar ou devolver usuário; verificar OTP).

Camada MVC	Onde está no código	Responsabilidade
Model	`backend/src/models/userModel.js`, `config/database.js`	Único ponto que executa SQL no MySQL.

Endpoints da API

Método e rota	Proteção	Entrada	Respostas
`POST /auth/sync`	Bearer token Firebase	`{ nome }` (opcional) + token	200 usuário existente · 201 usuário criado · 401 token ausente/inválido · 500 erro de banco
`POST /auth/verify-otp`	Pública	`{ email, codigo }`	200 código verificado · 400 código incorreto ou expirado · 500 erro interno

Estrutura de pastas

```
4-MVP/mvp/
├── backend/                      API REST (Node.js + Express)
│   ├── database.sql              criação do banco slowdown_db e da tabela users
│   ├── .env.example              modelo das variáveis de ambiente
│   └── src/
│       ├── server.js             ponto de entrada (porta 3000)
│       ├── routes/authRoutes.js  /auth/sync e /auth/verify-otp
│       ├── middlewares/          verifyFirebaseToken
│       ├── controllers/          authController (e humorController, ainda sem rota)
│       ├── models/userModel.js   acesso ao MySQL
│       └── config/               pool MySQL e Firebase Admin
└── frontend/                     App Flutter
    ├── lib/main.dart             inicializa Firebase e ProviderScope
    ├── lib/screens/              telas (View) e modelos persistidos localmente
    ├── lib/src/providers/        AuthNotifier (Riverpod)
    ├── lib/src/repositories/     AuthRepository
    ├── lib/src/services/         ApiConfig, dioProvider, SessionManager, AuthService
    ├── lib/src/models/           UsuarioModel
    ├── lib/src/utils/            validadores (regras de negócio puras)
    └── test/                     testes de unidade e de widget
```

Tecnologias

Camada	Tecnologia (versão no projeto)
App	Flutter / Dart ^3.12, flutter_riverpod ^3.3, dio ^5.9, http ^1.6, shared_preferences ^2.5, firebase_core ^4.11, firebase_auth ^6.5, google_sign_in ^6.2, spotify_sdk ^3.0
API	Node.js 18+, express ^5.2, firebase-admin ^14.0, mysql2 ^3.22, cors, dotenv; nodemon em desenvolvimento
Banco	MySQL 8 (utf8mb4)
Serviços externos	Firebase Authentication, Google Sign-In, Spotify App Remote
Qualidade e gestão	flutter_test, flutter_lints; Git + GitHub (Git Flow, Pull Requests), GitHub Projects

Arquitetura planejada × implementada

Para manter a documentação fiel ao software, a tabela registra os itens previstos no TP2 que ainda não fazem parte do MVP.

Previsto no TP2	Situação no MVP
Gemini API no chatbot	Substituída por motor de regras local (`_LocalChatEngine`); integração prevista via endpoint próprio da API.
Stripe API (Premium)	Não integrada; status Premium salvo no aparelho.
Firebase Cloud Messaging	Não integrado; apenas as preferências de notificação são salvas.
Google Fit / Health Connect	Código preparado e desativado em `health_service.dart` ; BPM simulado.
Google Speech-to-Text	Não integrado; a preferência de comandos de voz é salva.
Persistência do humor no MySQL	`humorController.registrarHumor` existe, mas sem tabela nem rota; o humor é salvo no aparelho.

5. Padrões de Projeto

Padrões identificados no código-fonte, com o problema que cada um resolve e um trecho real do repositório.

5.1. MVC (Model-View-Controller) arquitetural

Problema que resolve: Separar interface, regras e acesso a dados para que mudanças em uma camada não afetem as outras.

Onde está: Rotas → middleware → `authController` (Controller) → `userModel` (Model); telas Flutter (View). O controller nunca executa SQL.

```
// controllers/authController.js
const usuarioExistente = await buscarPorFirebaseUid(uid);   // chama o Model
if (usuarioExistente) return res.status(200).json({ usuario: usuarioExistente });
const novoId = await criar({ firebaseUid: uid, nome, email });
```

5.2. Repository estrutural / acesso a dados

Problema que resolve: Esconder das telas de onde vêm os dados (HTTP, banco local) atrás de uma interface por domínio.

Onde está: `AuthRepository` encapsula a chamada `/auth/sync` e devolve um `UsuarioModel`; `EmotionalEntryStorage` faz o mesmo para os registros emocionais locais — a troca para o backend não exige mudar as telas.

```
class AuthRepository {
  final Dio _client;
  AuthRepository(this._client);
  Future<UsuarioModel> syncUsuario(String nomeOpcional) async {
    final response = await _client.post('/auth/sync', data: {'nome': nomeOpcional});
    return UsuarioModel.fromJson(response.data['usuario']);
  }
}
```

5.3. Injeção de Dependência (Provider) criacional

Problema que resolve: Evitar que as classes criem suas próprias dependências, facilitando troca e testes.

Onde está: Riverpod: `dioProvider` cria o cliente HTTP; `authRepositoryProvider` injeta esse cliente no repositório; `AuthNotifier` obtém o repositório por `ref.read`.

```
final authRepositoryProvider = Provider<AuthRepository>((ref) {
  return AuthRepository(ref.watch(dioProvider));   // dependência injetada
});
```

5.4. Observer comportamental

Problema que resolve: Atualizar a interface automaticamente quando o estado muda, sem acoplamento direto.

Onde está: `AuthNotifier` (AsyncNotifier) publica estados *loading/data/error*; a `LoginScreen` observa com `ref.watch` e mostra o carregamento. Também usado por `setState` e `AnimationController` nas telas.

```
// login_screen.dart
final authState = ref.watch(authNotifierProvider);   // inscreve a tela
final isLoadingLocal = authState.isLoading;
```

5.5. Chain of Responsibility (Middleware e Interceptor) comportamental

Problema que resolve: Aplicar etapas transversais (autenticação, cabeçalhos) antes do tratamento da requisição.

Onde está: No servidor, `verifyFirebaseToken` valida o token e chama `next()` ou encerra com 401. No app, o interceptor do Dio adiciona o token em toda requisição.

```
router.post('/sync', verifyFirebaseToken, syncUser);

// api_client.dart
onRequest: (options, handler) async {
  final token = await SessionManager.obterToken();
  if (token != null) options.headers['Authorization'] = 'Bearer $token';
  return handler.next(options);
}
```

5.6. Singleton criacional

Problema que resolve: Garantir uma única instância de recursos caros ou globais.

Onde está: Pool de conexões MySQL e app Firebase Admin exportados por módulo (o Node mantém uma única instância em cache); no app, `FirebaseAuth.instance` e `SharedPreferences.getInstance()`.

```
// config/database.js
const pool = mysql.createPool({ /* ... */ connectionLimit: 10 });
module.exports = pool;   // todos os require() recebem o mesmo pool
```

5.7. Factory Method (construtores nomeados) criacional

Problema que resolve: Criar objetos válidos de forma expressiva, escondendo a regra de montagem.

Onde está: `AuthResult.sucessoCom()` / `AuthResult.erro()`, `ResultadoBpm.aceito()` / `rejeitado()`, `UsuarioModel.fromJson()`, `EmotionalEntry.fromJson()`.

```
factory ResultadoBpm.aceito(StatusBpm status) => ResultadoBpm._(
      valido: true, status: status, gerarAlerta: ValidadorBpm.gerarAlerta(status));
factory ResultadoBpm.rejeitado(String mensagem) => ResultadoBpm._(
      valido: false, gerarAlerta: false, mensagemErro: mensagem);
```

5.8. Strategy comportamental

Problema que resolve: Variar um algoritmo sem alterar quem o usa.

Onde está: Cada `BadgeInfo` recebe sua condição de desbloqueio como função; `GamificationStatus` apenas percorre o catálogo. O `ChatbotStyle` escolhe a estratégia de resposta do assistente.

```
BadgeInfo(id: 'streak_7', title: 'Sequência de 7 dias',
          unlockCondition: (s) => s.streakDays >= 7),
// ...
for (final badge in kBadgeCatalog) {
  if (badge.unlockCondition(this)) unlockedBadgeIds.add(badge.id);
}
```

5.9. Facade estrutural

Problema que resolve: Oferecer uma interface simples para um conjunto de chamadas complexas.

Onde está: `SessionManager` esconde as chaves do SharedPreferences; `AuthService.loginComGoogle()` reúne Google Sign-In, Firebase, API e sessão em um único método usado pelas telas.

```
static Future<void> salvarSessao({required String token,
    required int usuarioId, required String usuarioNome}) async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.setString(_chaveToken, token);
  await prefs.setInt(_chaveUsuarioId, usuarioId);
  await prefs.setString(_chaveUsuarioNome, usuarioNome);
}
```

6. Prototipação das Telas

O protótipo de alta fidelidade foi desenhado no Figma e implementado em Flutter. As capturas abaixo foram feitas do aplicativo em execução (pasta `4-MVP/prints`), e a Figura 18 mostra como as telas se ligam.

Identidade visual: amarelo `#F5B800` na barra superior e botões, fundo em degradê creme (`#F5F0A0` → `#E8E4A0`), texto escuro `#1C1C1C`, logotipo “SLOW DOWN” e o gato-mascote.

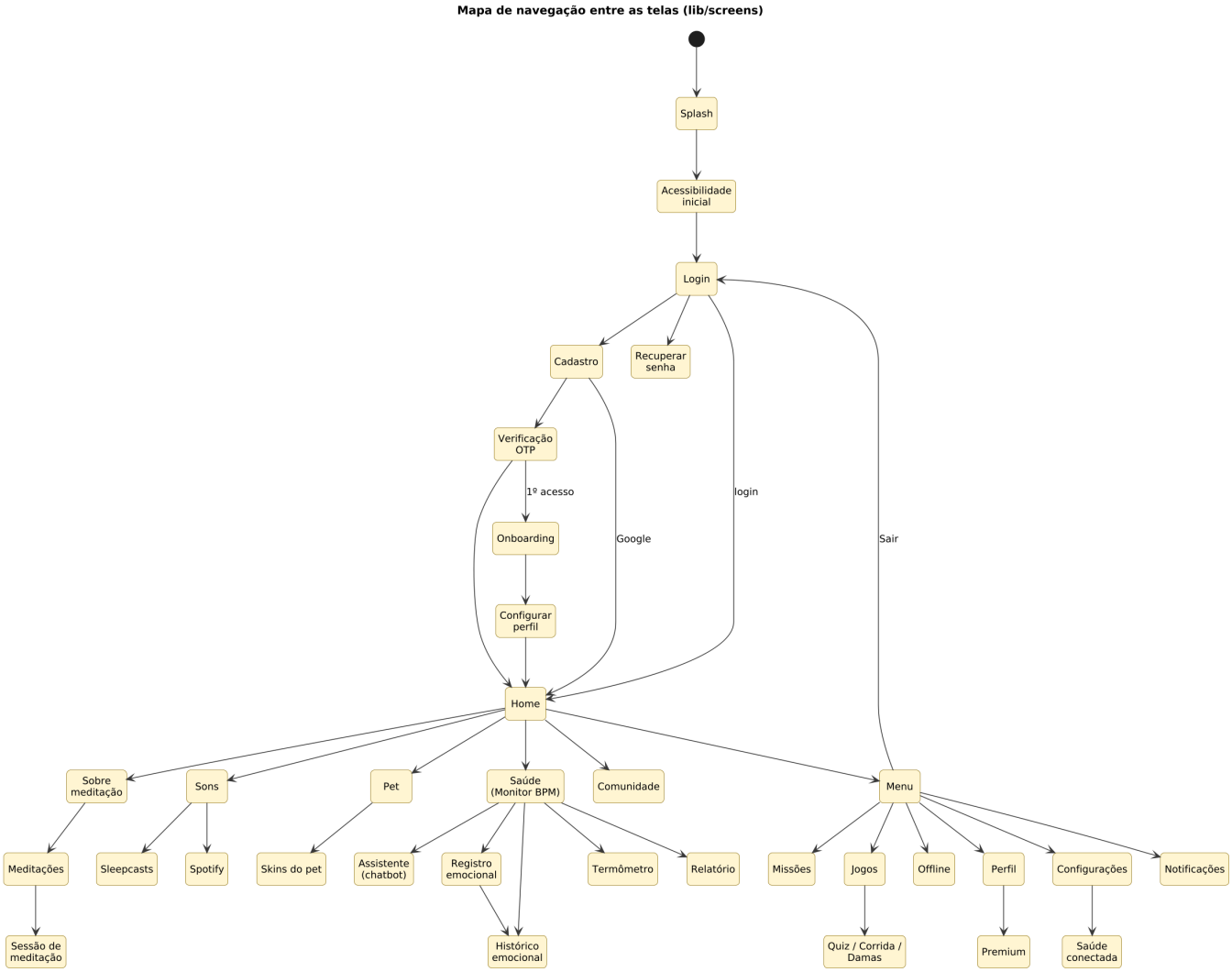
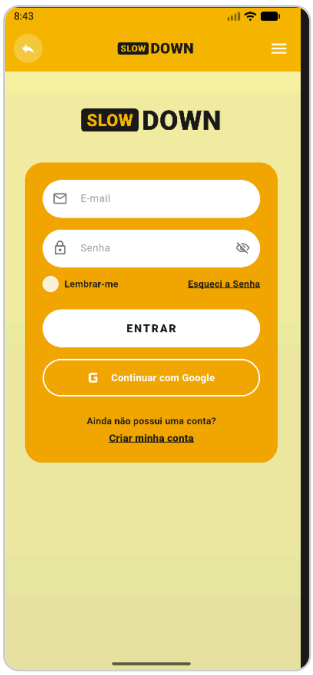


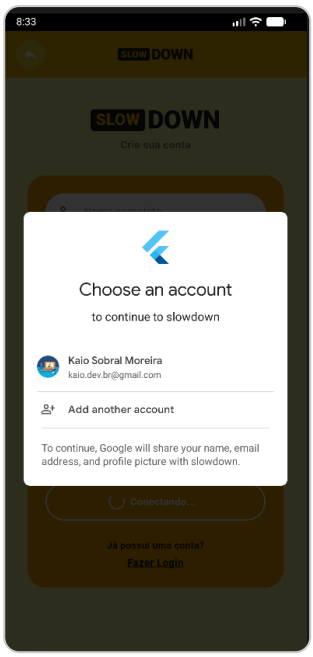
Figura 18 — Mapa de navegação entre as telas do aplicativo.



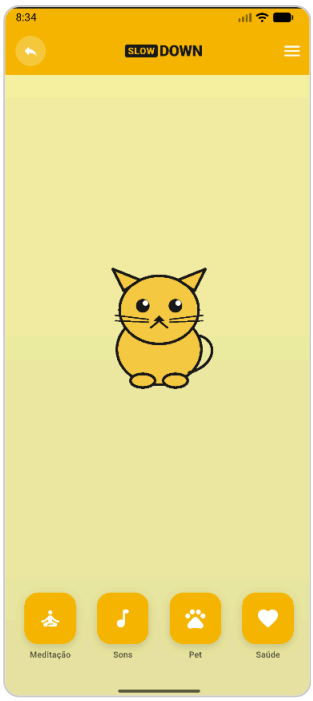
Login (UC04, UC05)



Cadastro (UC02)



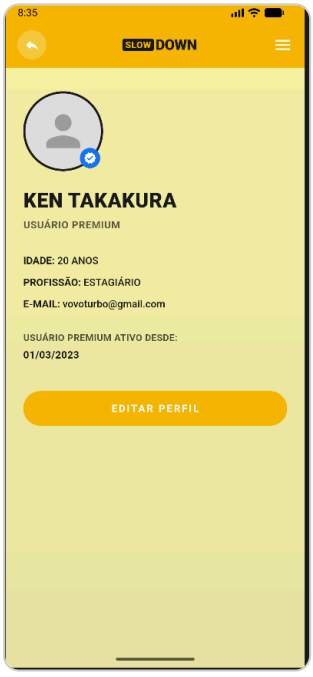
Cadastro com Google (UC05)



Home — Meditação, Sons, Pet e Saúde

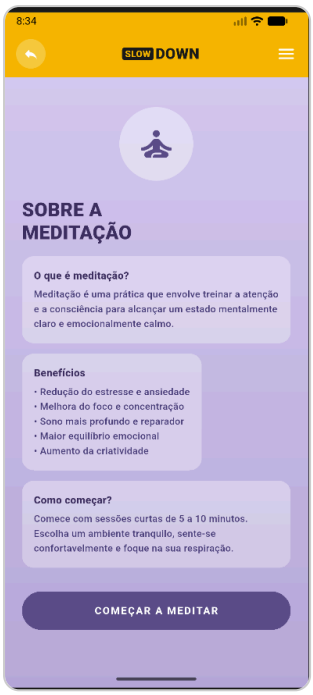


Menu (UC08, UC10, UC13)

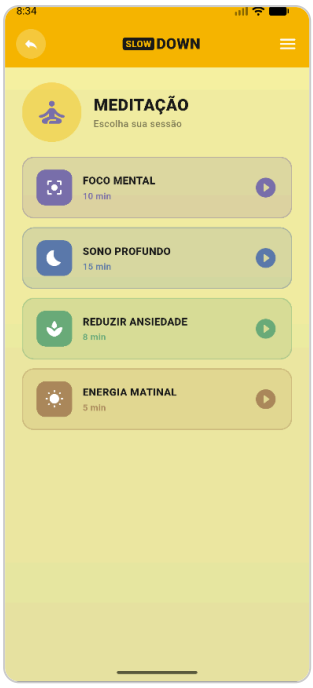


Perfil (UC08)

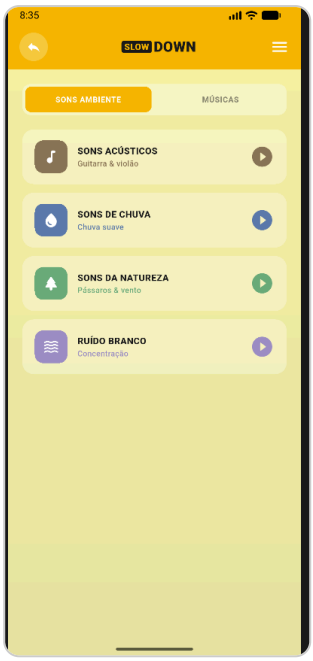
Figura 19 — Telas de acesso, Home, Menu e Perfil. A captura do Menu é anterior à inclusão dos botões Notificações, Missões, Jogos e Offline.



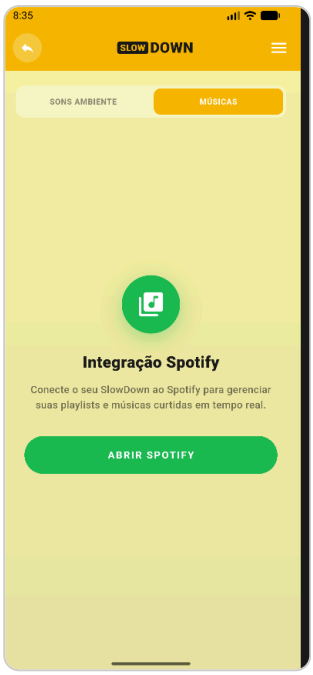
Sobre a meditação (UC20)



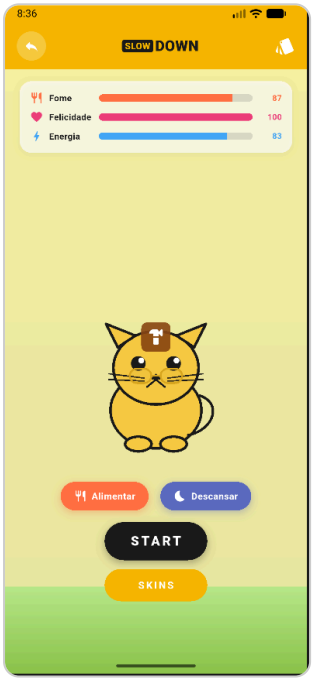
Escolha da sessão (UC20)



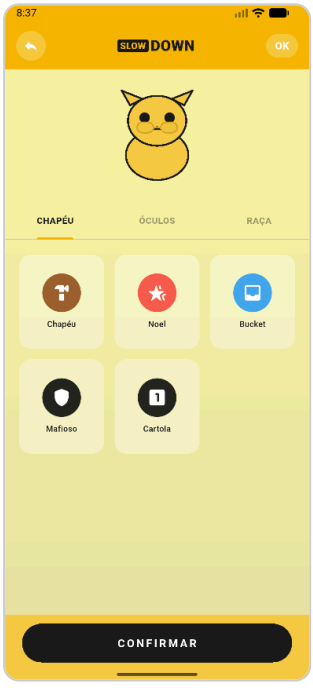
Sons ambiente (UC21)



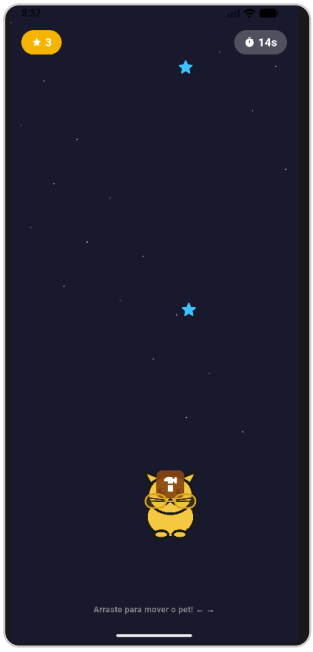
Integração Spotify (UC22)



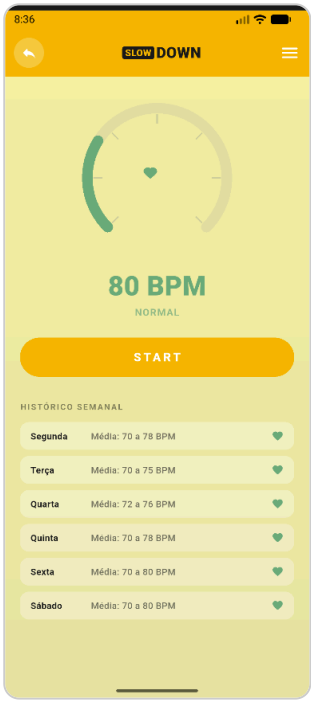
Pet virtual (UC24)



Personalizar pet (UC25)



Minijogo do pet (UC24)



Monitor de BPM (UC17)

Figura 20 — Telas de relaxamento, pet e monitoramento.

Apêndice A — Gerência de configuração e versionamento

O código e esta documentação são itens de configuração versionados no GitHub (`github.com/kaiosdev/projeto-pratico-es`). A equipe adota o **Git Flow** apresentado na disciplina:

Branch	Papel	Recebe merge de
`main`	Versão entregue (baseline); cada entrega recebe uma tag (ex.: `v1.0.0`).	`release` (Pull Request)
`release`	Ambiente de homologação da próxima entrega.	`development` (Pull Request)
`development`	Integração contínua das funcionalidades.	`feature/*` (Pull Request)
`feature/<nome>`	Uma mudança isolada (funcionalidade, correção, documentação). Apagada após o merge.	—

Commits semânticos, pequenos e descritivos, no formato `:emoji: tipo(escopo): descrição`, por exemplo:

```
:sparkles: feat(menu): adiciona acesso às telas de missões, jogos e offline
:bug: fix(menu): encerra sessão do Firebase ao sair do app
:recycle: refactor(monitor): usa ValidadorBpm para classificar a frequência cardíaca
:memo: docs(documentacao): adiciona documentação do sistema v2.0 em PDF
:see_no_evil: chore(gitignore): ignora dependências e artefatos de build
```

O arquivo `.gitignore` impede o envio de credenciais (`.env`, chave do Firebase Admin) e de arquivos gerados (`node_modules/`, `build/`, `.dart_tool/`). As regras completas estão em `CONTRIBUTING.md` e o modelo de Pull Request em `.github/pull_request_template.md`.